G. PULLAIAH COLLEGE OF ENGINEERING AND TECHNOLOGY
Autonomous · Affiliated to JNTUA, Anantapur · Kurnool

A40531 · DEEP LEARNING · IV B.Tech CSE · I Semester

MID-2

Units III · IV · V

Model Answers for the 10-Mark Question Bank

Regularisation · optimisation · CNNs · recurrent networks

UNIT III
Regularisation and Optimisation
overfitting · penalties
dropout · optimisers
answers 1 – 8

UNIT IV
Convolutional Networks
kernels · strides
pooling · priors
answers 9 – 16

UNIT V
Recurrent Networks, LSTM and Autoencoders
unrolling · gates
seq2seq · autoencoders
answers 17 – 24

Every answer carries a numbered derivation, a vector figure, an examiner's checklist and a one-line golden statement.
Prepared for the Mid-2 examination · A40531 Deep Learning · 2025–26

G. Pullaiah College of Engineering and Technology, Kurnool · Department of CSE (Data Science)

24	3	20	100
MODEL ANSWERS	UNITS	FIGURES	SYLLABUS MARKS

Contents

Twenty-four ten-mark answers, grouped by unit. Page numbers refer to this document.

UNIT III — Regularisation and Optimisation . . . 3

Q1. A regularisation plan for severe overfitting . . . 3

Q2. L1 versus L2 penalties and sparsity . . . 6

Q3. Dropout strategy, effective ensembles and compute cost . . . 8

Q4. Norm constraint and its relation to the penalty term . . . 10

Q5. Optimiser sequence for unstable training . . . 12

Q6. Why deep-network optimisation differs from pure optimisation . . . 14

Q7. AdaGrad versus RMSProp versus Adam . . . 16

Q8. Early stopping, dropout and augmentation . . . 18

UNIT IV — Convolutional Neural Networks . . . 20

Q1. CNN design: layers, kernels, strides and pooling . . . 20

Q2. Kernel, stride and padding: output size and cost . . . 22

Q3. FC versus CNN: sharing, locality, scalability . . . 24

Q4. Convolution and pooling as architectural priors . . . 26

Q5. Pooling where localisation matters . . . 28

Q6. Multiclass CNN pipeline: flatten versus GAP . . . 29

Q7. Underfitting caused by pooling . . . 32

Q8. Justifying a CNN architecture for a real application . . . 33

UNIT V — Recurrent Networks and Autoencoders . . . 36

Q1. Unfolded simple RNN and parameter sharing . . . 36

Q2. Vanishing and exploding gradients . . . 38

Q3. Standard vs bidirectional vs deep RNN vs LSTM . . . 40

Q4. LSTM gates and the internal state . . . 42

Q5. Strategy for long sequences . . . 44

Q6. Encoder–decoder seq2seq . . . 46

Q7. Autoencoder for dimensionality reduction . . . 48

Q8. Autoencoders and representation learning . . . 50

UNIT III

Regularisation and Optimisation

Overfitting, penalty terms, dropout, constraints and the optimiser ladder

UNIT AT A GLANCE

#	Question	Open with
1	A regularisation plan for severe overfitting	Diagnose the gap, then intervene in the order data → loss → architecture → optimiser.
2	L_1 versus L_2: why only one of them is sparse	The gradient of the L_1 penalty is constant in magnitude, so it can drive a weight to exactly zero.
3	Dropout as a training strategy	A random mask per mini-batch; test-time scaling turns 2^n thinned nets into one geometric-mean ensemble.
4	Norm constraint and its Lagrangian twin	A hard constraint on the norm and a penalty term are the same problem linked by the KKT multiplier.
5	An optimiser sequence for training that will not converge	Fix initialisation first, then momentum, then RMSProp, then Adam — and say why each rung helps.
6	Why deep-network optimisation is not pure optimisation	The objective is a surrogate for generalisation, the landscape is non-convex, and gradients are sampled.
7	AdaGrad, RMSProp and Adam compared	Same adaptive step; different memory of past squared gradients.
8	Early stopping, dropout and augmentation trade-offs	Three regularisers at three different levels — steps, activations, data — with different costs.

Q1 Designing a regularisation plan when a deep network overfits severely

CO3 · Apply–Analyse · 10 marks · ~3 pages

What "severe overfitting" looks like

Overfitting is present when the model has memorised the training set: the training loss keeps falling while the validation loss bottoms out and then rises. The quantitative signal is the **generalisation gap** between the two curves, and the diagnostic is the pair of learning curves. Before prescribing anything, decide which of the two failures you have, because the remedies are opposite in direction:

Sign	What the curves show	First move
high variance	training loss near zero, validation loss far above it and rising	regularise, get more data, simplify
high bias	both losses high and flat	bigger model, train longer, better features
noisy labels	the training loss itself is high, validation close to it	clean labels, robust loss

The question assumes severe overfitting, i.e. a large gap. The plan below attacks it in a fixed order: **data first, then the objective, then the architecture, then the optimiser**. Cheap, low-risk interventions come before expensive ones.

Step 1 — Enlarge the effective data

- **Collect more data.** Only this changes the variance term directly; ten times more data beats any hyper-parameter.
- **Augment** with label-preserving transforms — crops, flips, small rotations, colour jitter for images; time warping, noise injection for audio; token dropout, back-translation for text.
- **Synthetic oversampling** for rare classes, used carefully because it can add noise instead of information.

The theoretical statement is the decomposition

$$J(\theta) = \text{bias}^2 + \text{variance} + \sigma_\varepsilon^2 \tag{3.1}$$

and variance falls roughly as $O(1/m)$ in the number of examples m. Nothing else in this plan has that guarantee.

Step 2 — Change the objective: add a penalty

Train on the regularised objective instead of the plain empirical risk:

$$\tilde{J}(\theta) = J(\theta) + \lambda \Omega(\theta), \qquad \Omega(\theta) = \frac{1}{2}\|\theta\|_2^2 \tag{3.2}$$

which for plain gradient descent changes the update from a plain step to

$$\theta \leftarrow (1 - \eta\lambda)\,\theta - \eta \nabla J(\theta) \tag{3.3}$$

so every step multiplies the weights by a factor slightly below one — **weight decay**. The penalty is chosen to match the prior you believe: L_2 for "weights should be small and spread", L_1 for "few features matter", elastic net when features are correlated. Choose λ on a logarithmic grid by cross-validation; the curve of validation loss against $\log \lambda$ is usually flat-bottomed and U-shaped.

Step 3 — Regularise inside the architecture

- **Dropout** on the dense layers (rate 0.3–0.5), and lower rates on convolutional layers; it is the cheapest way to stop co-adaptation.
- **Batch normalisation**, which changes the effective objective and usually allows a higher learning rate; it is a mild regulariser because of the mini-batch noise.
- **Early stopping** on the validation loss: keep the weights at the best epoch.
- **Reduce capacity** only if the gap survives everything above: fewer layers, fewer units, or a smaller kernel budget.
- **Weight tying / sharing** where the problem allows it, e.g. convolutional weights, siamese towers.

Step 4 — Make the optimiser less greedy

Smaller mini-batches inject gradient noise, which is itself a regulariser; a decaying learning-rate schedule lets the model settle into flat regions; label smoothing replaces one-hot targets with reduced confidence and prevents over-confident logits. All three reduce the gap without touching the data or the architecture.

Step 5 — Average models

Snapshot ensembles, checkpoint averaging and multi-seed ensembles reliably buy one to three points of accuracy at the cost of inference time. They are the last resort because they do not fix a broken training procedure, they average over it.

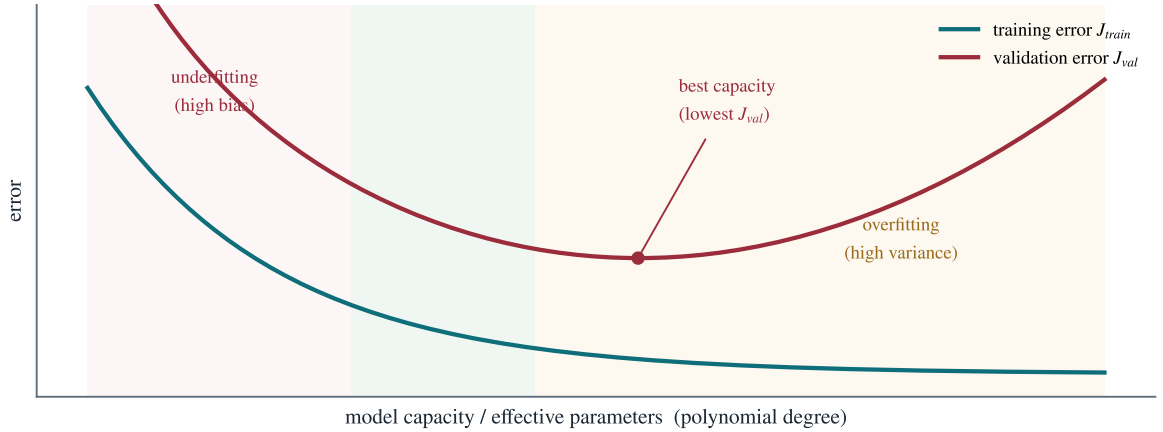


Fig. 3.1 Validation error is U-shaped in capacity; regularisation moves the minimum to the left, more data lowers the whole curve.

Choosing the strength — a worked protocol

1. Hold out a validation split (or use k-fold for small data).
2. Fix everything else and sweep one knob at a time in the order data → penalty → dropout → early stopping.
3. Record both losses and the gap for each setting; stop when the gap is small and the validation loss has stopped improving.
4. Re-check on the untouched test set exactly once.

Intervention	Cost	Typical effect on the gap
more/augmented data	data collection or CPU per epoch	largest, most durable
L_2 penalty	one extra term in the gradient	1–3 points, monotone in the strength
dropout	2–3× epochs to converge	large on dense nets
early stopping	needs a validation split	free, prevents the tail of the curve
smaller model	capacity	can convert variance into bias
ensembling	inference time	1–3 points, no training cost

THE ORDER MATTERS

Do not start with "use dropout". If the gap is caused by 500 labelled images and a 20-million-parameter backbone, the honest first answer is *more data or a smaller backbone*; dropout applied to an already-underfitting model will simply make both curves worse. State your diagnosis, then the intervention, then the measurement that tells you it worked.

GOLDEN LINE

A regularisation plan is a diagnosis followed by an ordered list of interventions — data, objective, architecture, optimiser, ensembling — each with a predicted effect and a way to measure it.

EXAMINER'S CHECKLIST

- Names the symptom quantitatively (the gap between the two losses, learning curves).
- Gives at least four levels of intervention and justifies the ordering.
- Writes the regularised objective and the weight-decay update.
- Mentions that more data attacks variance with an $O(1/m)$ guarantee.
- Ends with a measurement protocol: validation split, one knob at a time, test set used once.

KEYWORDS TO WRITE IN THE EXAM
generalisation gap · weight decay · data augmentation · dropout · early stopping · capacity · bias–variance decomposition · validation protocol · ensemble averaging

Q2 Compare L_1 and L_2 penalties and explain why only L_1 produces sparse solutions

CO3 · Understand–Analyse · 10 marks · ~3 pages

The two penalties

Both add a term to the empirical risk that penalises large weights, differing only in the norm used:

$$\tilde{J}_2(\theta) = J(\theta) + \frac{\lambda}{2}\|\theta\|_2^2 = J(\theta) + \frac{\lambda}{2}\sum_i \theta_i^2 \tag{3.4}$$

$$\tilde{J}_1(\theta) = J(\theta) + \lambda\|\theta\|_1 = J(\theta) + \lambda\sum_i |\theta_i| \tag{3.5}$$

L_2 is called **ridge** regression in the linear case and **weight decay** in a network; L_1 is **lasso**. The corresponding gradients are qualitatively different:

$$\nabla_\theta \tilde{J}_2 = \nabla J + \lambda\theta, \qquad \nabla_\theta \tilde{J}_1 = \nabla J + \lambda\,\mathrm{sgn}(\theta) \tag{3.6}$$

Why L_2 shrinks but L_1 selects

The L_2 gradient contribution is proportional to the weight, so it becomes weaker as the weight approaches zero and the weight only decays geometrically. In the scalar case with no data term, the differential equation gives

$$\theta(t) = \theta_0\, e^{-\lambda t} \tag{3.7}$$

which is asymptotic to zero and never equal to it.

The L_1 gradient contribution is $\lambda\,\mathrm{sgn}(\theta)$: a **constant force of size** λ directed towards zero, independent of how small the weight already is. So whenever the data gradient at a weight is smaller in magnitude than λ, the total gradient points at zero and the weight is driven there in finite time and **stays exactly at zero**. This is the analytical reason for sparsity. The closed form for a single weight with quadratic data term and target y is the soft-threshold

$$w^\star = \mathrm{sign}(y)\,\max\left(|y| - \lambda,\ 0\right) \tag{3.8}$$

which is exactly zero for every $|y| \le \lambda$ — an entire interval of data values mapped to a zero weight.

The geometric explanation

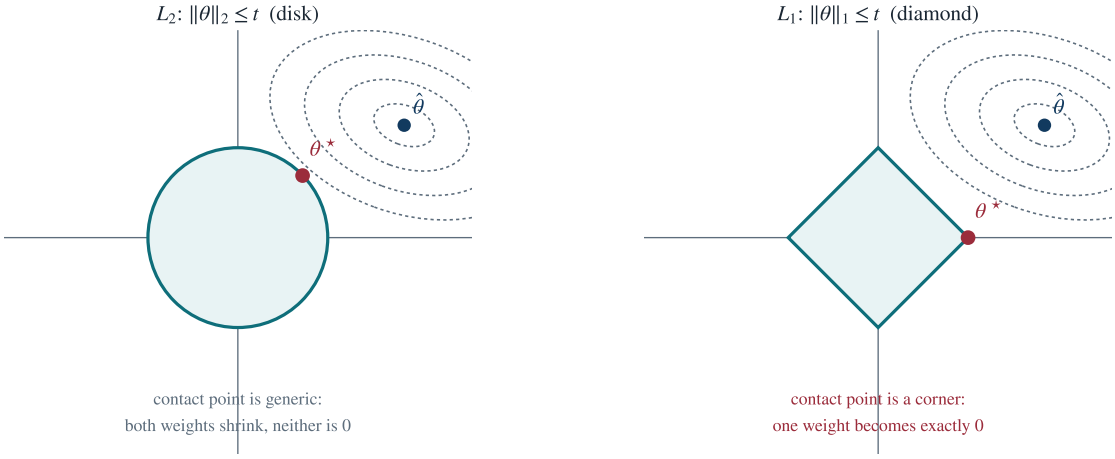


Fig. 3.2 Constraint geometry: the L_2 ball meets the loss contours at a generic point, the L_1 diamond meets them at a corner where one coordinate is zero.

Because the loss contours are smooth ellipses and the constraint set is a polytope with sharp corners on the axes, the first contact happens at a corner with probability one in high dimension. A corner of the L_1 ball has many zero coordinates; a point on the L_2 sphere has none. This is the same statement as the KKT condition of the next answer: at the solution the gradient of the loss must be a non-negative multiple of a subgradient of the norm, and only the L_1 norm has subgradients with zero components.

Ridge has a closed form; lasso does not

For a linear model with design matrix X and targets y,

$$\hat{\theta}_{\text{ridge}} = (X^\top X + \lambda I)^{-1} X^\top y \tag{3.9}$$

The added λI makes the matrix invertible even when $X^\top X$ is singular (more features than samples), which is why ridge is the standard fix for collinearity. Lasso has no closed form but is solved by coordinate descent or proximal gradient, each coordinate being a soft-threshold update. In a network, L_1 is implemented by the same subgradient step or by the proximal operator applied after each update.

Side-by-side comparison

Property	L_2 penalty	L_1 penalty
Solution	dense, all weights small	sparse, many exact zeros
Effect on correlated features	shares weight between them	picks one, zeroes the rest
Differentiability	smooth everywhere	non-smooth at zero (subgradient)
Closed form (linear model)	yes, a matrix inverse	no, needs an iterative solver
Shrinkage profile	proportional	constant, then hard zero
Typical use	default in deep nets, weight decay	feature selection, compression
Stability	stable, unique	can be unstable with correlated inputs

The compromise: elastic net

$$\Omega(\theta) = \lambda_1 \|\theta\|_1 + \lambda_2 \|\theta\|_2^2 \tag{3.10}$$

The L_2 part stabilises the solution when features are correlated — it makes the problem strictly convex and groups correlated features — while the L_1 part retains selection. In practice a small L_2 term alongside L_1 is almost always worth it.

PRACTICAL NOTE FOR NETWORKS

In a deep network plain L_2 on all weights is the default (10^{-4} is a common starting value). Applying L_1 to a whole network rarely gives useful sparsity at the level of individual weights; structured sparsity (whole filters, whole channels, group lasso) is the version that actually shrinks models. Say this in the exam — it shows you know the difference between a textbook result and an engineering one.

GOLDEN LINE

L_2 penalises the square, so its pull vanishes at the origin and weights only shrink asymptotically; L_1 penalises the absolute value, so its pull is a constant λ that can hold a weight at exactly zero — sparsity is a consequence of the geometry, not of the algebra.

EXAMINER'S CHECKLIST

- Writes both objectives and both gradients.
- Explains the constant-magnitude gradient of the absolute value against the proportional gradient of the square.
- Gives the geometric corner argument.
- States the closed form for ridge and notes that lasso needs a solver.
- Compares on sparsity, correlated features, smoothness and stability.

KEYWORDS TO WRITE IN THE EXAM

ridge · lasso · weight decay · subgradient · soft thresholding · convexity · feature selection · elastic net · polytope corner

Q3

Dropout as a regularisation strategy — mechanism, ensemble interpretation and computational cost

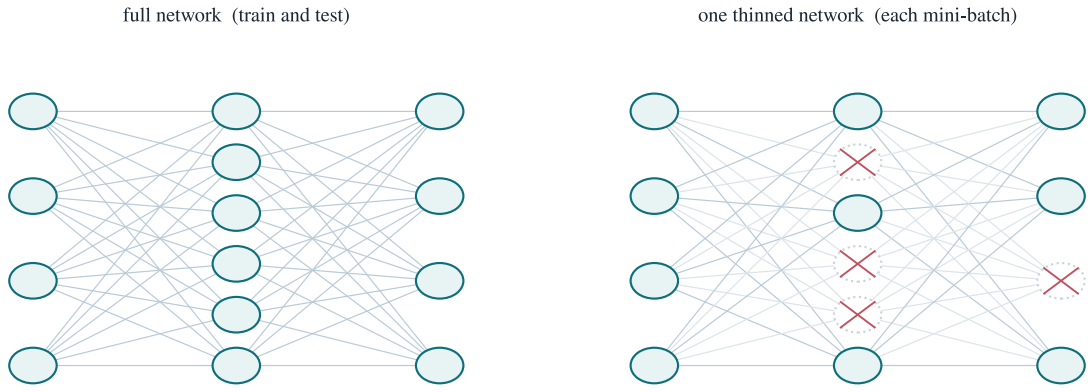
CO3 · Understand–Apply · 10 marks · ~3 pages

Mechanism

At every training step, each unit of a chosen layer is kept with probability q (the keep probability) and dropped with probability $1-q$, independently. With a mask drawn from a Bernoulli distribution the layer computes

$$\tilde{h} = m \odot h, \qquad m_i \sim \text{Bernoulli}(q) \tag{3.11}$$

and the mask is resampled for every mini-batch, so no unit can rely on a specific partner being present. The effect on the weights is that co-adaptation is broken: a feature must be useful in many random contexts, which is precisely a constraint against memorisation.



each unit survives with probability q; at test time weights are scaled by q — a geometric-mean ensemble of 2^n sub-networks

Fig. 3.3 Left: the full network. Right: one thinned network from a single mini-batch; the crossed units receive no gradient in that step.

Inference: how the ensemble is collapsed

Training samples 2^n different thinned sub-networks that share weights. At test time we want the average prediction of all of them, which is intractable. The standard approximation is to use the full network with the outgoing weights scaled by the keep probability:

$$W^{(l)}_{\text{test}} = q\, W^{(l)} \tag{3.12}$$

This matches the expected pre-activation of a linear unit, and for a softmax output it approximates the geometric mean of the sub-network distributions rather than the arithmetic mean — a well-known subtlety, but the approximation is good enough in practice and is what makes dropout free at inference.

The modern equivalent, **inverted dropout**, divides by q during training instead:

$$\tilde{h} = \frac{m \odot h}{q} \tag{3.13}$$

so that the expectation of the masked activation equals the unmasked one and nothing at all needs to change at test time. This is what every framework implements today.

Why this is an ensemble

Each mini-batch trains one member of a family of 2^n architectures that share parameters. The prediction of the scaled network is a single evaluation that approximates the model average, so dropout buys the variance reduction of ensembling without k times the inference cost. The analogy is exact enough to be examinable: *dropout is bagging of sub-networks with parameter sharing and one Monte-Carlo sample per step.*

Cost analysis

Cost component	During training	During inference
parameters	unchanged	unchanged
arithmetic per example	one mask multiply per unit (about 1–2%)	zero
memory	mask per layer per step (small)	zero
convergence	2–3× more epochs, higher learning rate needed	—
gradient noise	variance increases, scale by $1/q$	—
accuracy at test	—	free ensemble effect

The real cost is therefore **optimisation time**, not arithmetic: noisy gradients mean the loss curve is jagged, the effective learning rate must often be raised, and training may need two or three times as many epochs. In exchange, nothing is added to the deployed model.

A concrete dropout strategy

- Input layer: rate 0.1–0.2 (image pixels) — rarely more, it destroys signal.
- Convolutional layers: 0.1–0.25, or none if batch normalisation is already present.
- Fully connected layers: 0.3–0.5 — this is where most of the parameters and most of the memorisation live.
- Output layer: never.
- Always pair with early stopping on validation loss; the best epoch usually comes later than without dropout.
- When a batch-norm layer follows, note the variance mismatch between train and test: dropout changes the activation variance, so apply dropout after the normalisation, or use a dropout variant designed for it.

TWO COMMON MISTAKES

1. Applying the $1/q$ scaling at test time as well as training (double correction) — the network becomes under-confident.
2. Leaving dropout on during validation: predictions become random and the validation loss is meaningless. Always switch the model to inference mode before evaluating.

GOLDEN LINE

Dropout trains 2^n weight-sharing sub-networks in one pass and evaluates their geometric mean in one pass, so it delivers ensemble-level variance reduction at essentially zero inference cost, paying instead in training epochs.

EXAMINER'S CHECKLIST

- Writes the Bernoulli mask and the keep probability.
- Explains both scaling conventions and why inverted dropout removes test-time work.
- Connects to the 2^n ensemble and the geometric mean.
- Quantifies the cost: no new parameters, negligible arithmetic, 2–3× epochs.
- Gives layer-wise rates and mentions the interaction with batch normalisation.

KEYWORDS TO WRITE IN THE EXAM

co-adaptation · Bernoulli mask · keep probability · inverted dropout · thinned network · geometric mean · model averaging · variance reduction · inference cost

Q4 **Explain the norm constraint on parameters and derive its relation to the penalty term using the Lagrangian and KKT conditions**

CO3 · Apply–Analyse · 10 marks · ~3 pages

The two formulations

The regularised problem can be written in two apparently different ways. The **penalty** (soft) form adds a term to the loss:

$$\theta^\star = \arg\min_\theta J(\theta) + \lambda\Omega(\theta) \tag{3.14}$$

The **constraint** (hard) form restricts the weights to a region:

$$\theta^\star = \arg\min_\theta J(\theta) \quad \text{subject to} \quad \Omega(\theta) \le t \tag{3.15}$$

For the squared L_2 norm the feasible set is a ball of radius $\sqrt{2t}$; for the L_1 norm it is the diamond of the previous answer. The claim to be proved is that for every radius t there exists a $\lambda \ge 0$ giving the same solution, and conversely.

Lagrangian and the KKT system

Form the Lagrangian with multiplier $\lambda \ge 0$:

$$\mathcal{L}(\theta, \lambda) = J(\theta) + \lambda(\Omega(\theta) - t) \tag{3.16}$$

The KKT conditions for this convex problem are stationarity, dual feasibility, complementary slackness and primal feasibility:

$$\nabla_\theta J(\theta^\star) + \lambda\, \partial\Omega(\theta^\star) \ni 0 \tag{3.17}$$

$$\lambda \ge 0, \qquad \lambda(\Omega(\theta^\star) - t) = 0 \tag{3.18}$$

$$\Omega(\theta^\star) \le t \tag{3.19}$$

where the partial derivative symbol denotes the subdifferential, needed because the L_1 norm is not differentiable at zero. The second line is **complementary slackness**. It splits the answer into the two cases that the exam wants:

1. If the unconstrained minimiser already satisfies the constraint, the constraint is inactive and $\lambda = 0$: the solution is the plain maximum-likelihood estimate.
2. If the constraint is active, the radius is saturated and $\lambda > 0$: the stationarity condition is exactly the stationarity condition of the penalty problem with that λ.

Hence the constraint problem and the penalty problem have the same set of solutions, and the multiplier of the constraint problem *is* the regularisation coefficient of the penalty problem.

The one-to-one map between radius and strength

Define the achieved norm as a function of the penalty strength. Along the regularisation path this function is continuous and non-increasing: a larger λ shrinks the weights further, so the ball that contains the solution shrinks. This gives a monotone bijection on the range where the constraint is active, which is why the two formulations are called **equivalent** — equivalent as families, parameterised differently.

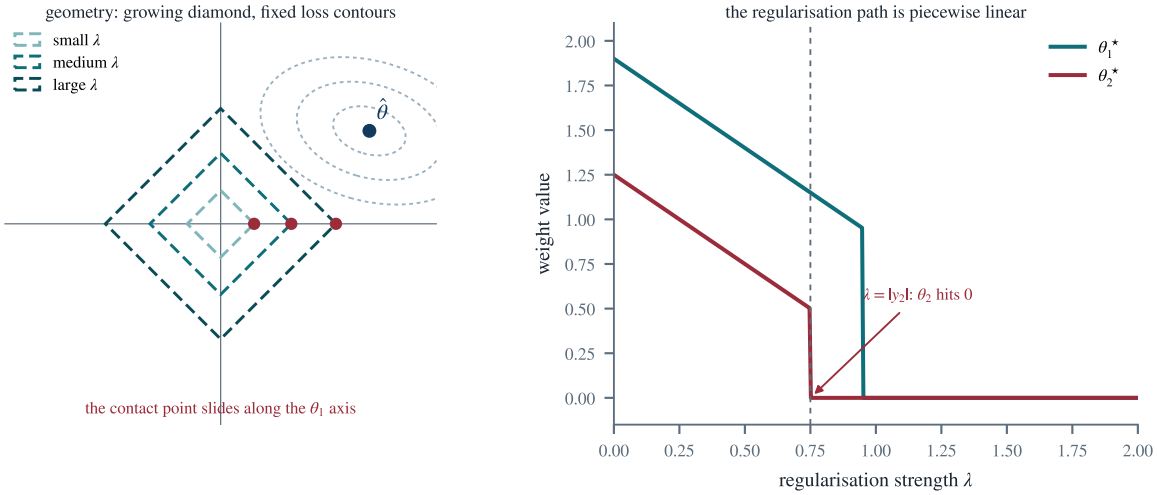


Fig. 3.4 The regularisation path: as λ grows the L_1 diamond grows relative to the loss contours and the solution slides along an axis, one weight at a time becoming exactly zero.

Worked example in one dimension

Take a quadratic data term and the absolute-value penalty. The constrained problem is to minimise $1/2(\theta - y)^2$ subject to $|\theta| \le t$. Its solution is the projection of y onto the interval $[-t, t]$:

$$\theta_c^\star(t) = \operatorname{sign}(y)\min(|y|, t) \tag{3.20}$$

The penalty problem gives the soft-threshold of the previous answer. Putting $t = |y| - \lambda$ makes the two expressions identical for $0 \le \lambda \le |y|$; for $\lambda > |y|$ the penalty solution is zero while the constraint solution is zero only if $t = 0$. The mapping between them is exactly the relation predicted by the theory, and at that point complementary slackness holds.

Why keep both views

Aspect	Constraint form	Penalty form
hyper-parameter	radius, interpretable as a trust region	strength, scale-dependent
optimisation	projection step after each update	gradient or subgradient term
tuning	hard to set without knowing the weight scale	straightforward log grid
geometry	explicit ball or diamond in weight space	implicit level sets
typical use	projected gradient, max-norm constraints, robustness	standard training in every framework

The constraint view is what makes the geometry obvious (why L_1 is sparse, why a max-norm constraint is a trust region); the penalty view is what is actually implemented, because a gradient term is cheaper than a projection.

LINK BACK TO UNIT I

This is the same KKT machinery used for the margin derivation of the support vector machine: there the constraint was that every point lies on the correct side of the margin with multipliers, and complementary slackness said that only support vectors have a positive multiplier. Here the constraint is on the norm of the parameter vector and the single multiplier plays the role of the whole multiplier vector; the statement "only the active constraints matter" is identical.

GOLDEN LINE

A norm constraint and a norm penalty are two parameterisations of one problem: the KKT multiplier of the active constraint is the penalty coefficient, and complementary slackness says the constraint only bites when the unconstrained optimum lies outside the ball.

EXAMINER'S CHECKLIST

- Writes both formulations and the Lagrangian with a non-negative multiplier.
- States stationarity, dual feasibility, complementary slackness and primal feasibility.
- Treats the inactive and active cases separately.
- Shows the monotone map between radius and strength and calls the families equivalent.
- Works the one-dimensional example through to the soft-threshold.

KEYWORDS TO WRITE IN THE EXAM

Lagrangian · KKT conditions · complementary slackness · active constraint · trust region · regularisation path · projection · subdifferential · equivalence of formulations

Q5 Training is unstable and the loss diverges — give the sequence of optimiser-side interventions

CO3 · Apply–Analyse · 10 marks · ~3 pages

First, be sure it is an optimiser problem

Divergence has three common causes that no optimiser fixes: a mis-scaled loss (forgetting to average over the batch), a bad learning rate, and unnormalised inputs. Check these first: print the loss on a single batch, confirm it decreases when you overfit that batch, and confirm the learning rate is not larger than the reciprocal of the largest curvature. The sequence below assumes the data pipeline is sane and the architecture is correct.

The order is deliberate: each rung of the ladder addresses a failure mode that the rungs below cannot fix, and each is cheaper to reason about than the next.

Rung 1 — Initialisation

A network with badly scaled initial weights has activations that either vanish or explode before the first update. Two standard choices, for a layer with n_{in} inputs and n_{out} outputs:

$$W_{ij} \sim \mathcal{N}\left(0, \frac{2}{n_{in}}\right) \quad \text{(He, for ReLU)} \tag{3.21}$$

$$W_{ij} \sim \mathcal{N}\left(0, \frac{1}{n_{in}}\right) \quad \text{(Xavier, for tanh)} \tag{3.22}$$

Both keep the variance of the activations constant across layers. The bias is initialised to zero (or a small positive value for ReLU to avoid dead units) and the last layer's weights are scaled down when using a softmax loss. Orthogonal initialisation for recurrent weights is the analogue for sequences.

Rung 2 — Stochastic gradient descent with momentum

The baseline that everything is compared against:

$$v_t = \mu v_{t-1} - \eta \nabla J(\theta_t), \qquad \theta_{t+1} = \theta_t + v_t \tag{3.23}$$

Momentum accumulates a running average of gradients, so it damps oscillation across steep directions and accelerates along shallow ones. It fixes **ill-conditioning** — the classic ravine — but it uses one global learning rate, so it cannot help when different parameters need different step sizes. If the loss still oscillates, reduce the learning rate; if it is flat for long stretches,

momentum is not your problem.

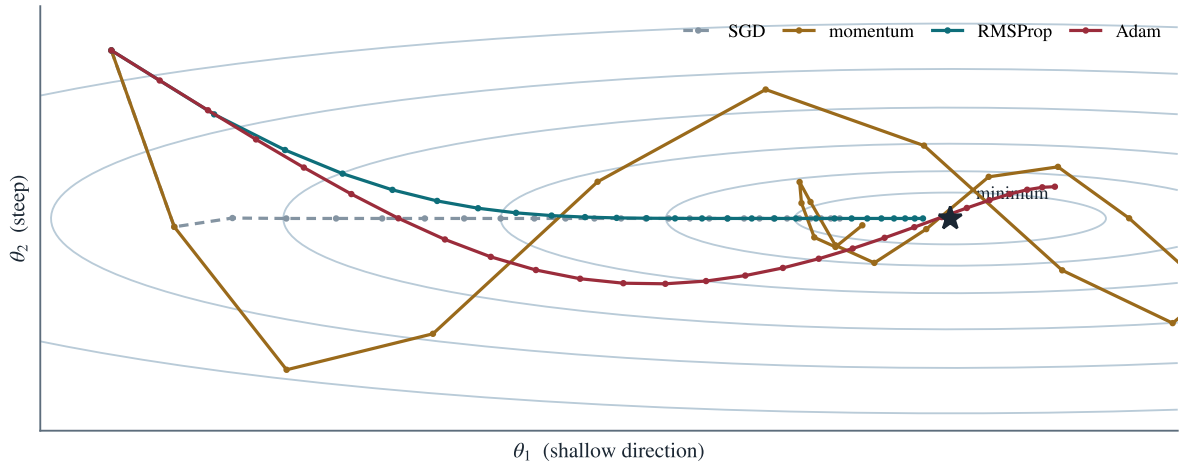


Fig. 3.5 Paths on an ill-conditioned quadratic. Plain descent zig-zags across the steep direction; momentum smooths it; the adaptive methods equalise the two directions.

Rung 3 — RMSProp

Divide each coordinate's step by the root of its recent squared gradient:

$$s_t = \rho s_{t-1} + (1-\rho) g_t^2, \qquad \theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{s_t} + \varepsilon} g_t \tag{3.24}$$

with a decay of about 0.9. A parameter with consistently large gradients gets a small effective step; a rare parameter with small gradients gets a large one. This is what fixes **per-parameter scaling**: embeddings, sparse features and biases no longer need hand-tuned rates.

Rung 4 — Adam

Adam adds momentum to the adaptive denominator and corrects the initialisation bias of both moving averages:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \qquad s_t = \beta_2 s_{t-1} + (1-\beta_2) g_t^2 \tag{3.25}$$

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \qquad \hat{s}_t = \frac{s_t}{1-\beta_2^t}, \qquad \theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{s}_t} + \varepsilon} \tag{3.26}$$

Defaults are $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$ and a step size of 10^{-3}. Adam is the composition of rungs 2 and 3 plus bias correction, which is why it is the last rung and not the first: if it fails you have no simpler fallback left.

Rung 5 — Guards around whichever optimiser you chose

1. **Gradient clipping** at a norm of one to five: rescale the gradient when its norm exceeds the threshold, $g \leftarrow g\, c / \|g\|$. Mandatory for recurrent nets, useful everywhere.
2. **Learning-rate warm-up** over the first few hundred steps so that the adaptive estimates can settle before large steps are taken.
3. **A decay schedule** — cosine or step decay — to finish training without oscillation.
4. **Loss-scale diagnostics**: log the gradient norm per layer; a single exploding layer localises the fault.

Symptom	Rung that fixes it	Why
loss becomes NaN in the first epoch	1, then 5	scale of initial activations, then clipping
oscillating loss, slow progress	2	momentum damps the cross-ravine component
some layers learn, others frozen	3	per-coordinate scaling
fast at first, then stuck	4	momentum plus adaptive denominator
periodic spikes late in training	5	step size too large near a minimum

HOW TO ANSWER THE "SEQUENCE" QUESTION

The examiner is testing whether you can order interventions by *cause*, not whether you can list optimisers. Write the ladder, and against each rung write the failure it repairs and the observable that tells you to climb to the next rung. Finish with the cheap guards — clipping and warm-up — because they are what makes Adam usable in practice.

GOLDEN LINE

Initialisation fixes scale, momentum fixes ill-conditioning, RMSProp fixes per-parameter scale, Adam combines the last two with bias correction, and clipping plus warm-up make any of them safe.

EXAMINER'S CHECKLIST

- Rules out data and learning-rate bugs before blaming the optimiser.
- Gives the He and Xavier variance formulas.
- Writes the update rule for each rung, including Adam's bias correction.
- States the failure each rung repairs.
- Adds clipping, warm-up and a schedule as guards.

KEYWORDS TO WRITE IN THE EXAM

divergence · weight initialisation · He scaling · momentum · adaptive learning rate · RMSProp · Adam · bias correction · gradient clipping · warm-up

Q6 Why is optimisation in deep learning different from pure optimisation?

CO3 · Understand–Analyse · 10 marks · ~3 pages

The formal statement of the problem

Training minimises the expected risk, which we cannot compute, so we minimise the empirical risk on a sample:

$$J(\theta) = \frac{1}{m}\sum_{i=1}^{m} L(f(x_i;\theta), y_i) + \lambda\Omega(\theta) \tag{3.27}$$

Pure optimisation asks for the minimiser of J, careless of what J is a proxy for. Deep learning asks for a parameter vector that **generalises**: the quantity we care about is the population risk, and the training objective is only an estimate of it. That single sentence generates most of the differences below.

Difference 1 — the objective is a surrogate

Minimising the training loss too well is harmful; the gap between population and training risk grows with capacity. Optimisation theory says "lower is better" at every step; deep learning says "lower on the validation set is better, and the training loss is a means". This is why early stopping, which *deliberately stops short of the minimum*, is a legitimate algorithm.

Difference 2 — the landscape is non-convex

A network's loss is non-convex in the parameters: permuting the units of a hidden layer gives the same function, so for k hidden units there are $k!$ equivalent minima, and similarly for scaling symmetries. Optimisation theory's convergence guarantees for convex problems do not transfer. Empirically, however, the picture is less grim than the theory suggests:

- **Local minima are rare in high dimension.** A critical point that is a local minimum in all n directions requires all n Hessian eigenvalues to be positive; for a random landscape in n dimensions most critical points are saddles, because a saddle needs only one negative eigenvalue.
- **Saddle points dominate.** Their number grows exponentially with n, and gradient descent spends most of its time in the neighbourhood of plateaus around them, where the gradient is small but the Hessian has both signs.
- **Good minima are connected.** Many distinct minima reach the same test accuracy; mode-connectivity studies show wide flat basins that are easy to reach by stochastic descent.

> **THE CONSEQUENCE FOR THE EXAM ANSWER**
> Say clearly: *the difficulty is not escaping bad local minima, it is the sheer number of saddle points and plateaus, plus the fact that we do not want the minimum of this particular function anyway.* This is the sentence most answers miss.

Difference 3 — gradients are stochastic

We never see the full gradient, only its estimate on a mini-batch of size B. The estimator is unbiased but has variance that decreases as B grows:

$$\mathbb{E}\left[\|\nabla \hat{J}_B - \nabla J\|^2\right] = \frac{\sigma^2}{B} \tag{3.28}$$

Noise acts as a regulariser (it prevents settling into sharp minima), allows escape from bad regions, and makes training scale with data. But it also means the loss curve is non-monotone and the stopping criterion must be statistical, not exact.

Difference 4 — scale and conditioning

Networks have between a million and a billion parameters, and the condition number of the Hessian can exceed 10^6: some directions are almost flat (unidentifiable combinations of weights) and some are extremely steep. First-order methods then need many steps, which is exactly the motivation for the adaptive methods of the previous answer. Parameter count also makes second-order methods (Newton, natural gradient) prohibitively expensive.

Difference 5 — the target is not stationary

In pure optimisation the data and the objective are fixed. In deep learning the distribution of the data may drift, the loss may be non-decomposable, and the model is retrained continuously. Added to this, the objective has a huge flat region, so "convergence" must be defined by validation performance rather than by the gradient norm falling below a threshold.

Aspect	Pure optimisation	Deep learning
what is minimised	the given objective	a sample estimate of it
landscape	convex by assumption	non-convex, symmetry, saddles
gradient	exact	mini-batch estimate with variance
dimension	moderate	10^6–10^9
stopping rule	gradient norm below a threshold	validation loss stops improving
success metric	distance to the optimum	test error on unseen data
local minima	the danger	rare and usually benign

GOLDEN LINE

Deep learning optimises a surrogate on a sampled, non-convex, extremely ill-conditioned surface, so a good optimiser is judged by generalisation, not by how close it gets to the minimum of the training loss.

EXAMINER'S CHECKLIST

- States that the training loss is a proxy for the population risk.
- Mentions non-convexity and the symmetry of the parameterisation.
- Explains that saddles and plateaus, not local minima, are the practical obstruction.
- Treats stochasticity as both a cost and a regulariser, with the variance formula.
- Contrasts stopping rules and success metrics in a table.

KEYWORDS TO WRITE IN THE EXAM

empirical risk · surrogate objective · non-convexity · saddle point · plateau · mini-batch noise · ill-conditioning · parameter symmetry · generalisation gap

Q7 Compare AdaGrad, RMSProp and Adam and justify which one you would choose

CO3 · Understand–Apply · 10 marks · ~2.5 pages

The common idea

All three replace the single global learning rate with a per-parameter step that is divided by a history of that parameter's gradients. Writing g_t for the gradient at step t and s_t for the accumulator, they differ only in how the accumulator is formed and whether a momentum term is added.

AdaGrad

$$s_t = s_{t-1} + g_t^2, \qquad \theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{s_t + \varepsilon}} g_t \tag{3.29}$$

The accumulator is a **sum over all past steps**, so it grows without bound and the effective rate decays monotonically. Parameters that have received large gradients are slowed permanently; parameters with rare, informative gradients keep a usable rate. The decay is roughly $\eta/\sqrt{t}$ for a stationary gradient, which is why AdaGrad is provably good for convex problems with sparse gradients. The same property is its weakness: in a long training run the accumulator becomes so large that learning effectively stops before the model has converged — *the learning rate dies*.

RMSProp

$$s_t = \rho s_{t-1} + (1-\rho) g_t^2, \qquad \theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{s_t + \varepsilon}} g_t \tag{3.30}$$

RMSProp replaces the sum by an **exponential moving average** with decay around 0.9, so the accumulator has an effective memory of about ten steps. Old gradients are forgotten, the denominator stays bounded, and learning continues indefinitely. It was designed exactly to fix the stall of AdaGrad, and it works well on non-stationary objectives such as mini-batch training.

Adam

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \qquad s_t = \beta_2 s_{t-1} + (1-\beta_2) g_t^2 \tag{3.31}$$

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \qquad \hat{s}_t = \frac{s_t}{1-\beta_2^t}, \qquad \theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{s}_t} + \varepsilon} \tag{3.32}$$

Adam is RMSProp plus **momentum on the numerator** plus **bias correction**. The bias correction matters: since the moving averages start at zero, the raw average is biased towards zero by a factor that is severe for the first few dozen steps — with a decay of 0.999 the factor is 0.001 at the first step. Dividing by the correction factor makes the estimate unbiased and allows a default step size of 10^{-3} to work from step one.

Comparison

AdaGrad	RMSProp	Adam	
accumulator	sum of squares	moving average of squares	moving averages of gradient and square
memory of past	infinite	about ten steps	hundreds to thousands of steps
momentum	no	no	yes, bias-corrected
behaviour late in training	stalls	steady	steady
extra memory	one scalar per parameter	one scalar	two scalars
hyper-parameters	step size	step size, decay	step size, two decays, epsilon
best for	sparse features, convex, short runs	non-stationary, recurrent nets	default for most deep nets
known weakness	premature stall	needs a tuned decay	can generalise slightly worse than plain momentum

Which to choose, and why

1. **Default:** Adam with a step size of 10^{-3}, because it is robust to the scale of each parameter and converges quickly on most architectures; it is the right first attempt on an unfamiliar problem.
2. **Sparse or bag-of-words style inputs with a convex model:** AdaGrad, where its long memory is an advantage rather than a stall.
3. **Recurrent networks and non-stationary objectives:** RMSProp or Adam; RMSProp's single accumulator is cheaper and sometimes more stable on long sequences.
4. **When the last fraction of accuracy matters (vision):** stochastic descent with momentum and a cosine schedule often generalises better than Adam, because the per-parameter scaling can lead it into sharper minima.

A DEFENSIBLE EXAM ANSWER

State the update rules, then reduce all three to one sentence: *AdaGrad divides by the root of the accumulated squared gradient, RMSProp divides by the root of a decaying average of it, and Adam also averages the gradient itself and corrects for the zero-initialisation bias.* Then justify a choice by the property that matters: sparsity → AdaGrad, non-stationarity → RMSProp, general default → Adam.

GOLDEN LINE

The three algorithms are one family: Adam is RMSProp plus momentum plus bias correction, and RMSProp is AdaGrad with a forgetting factor; the choice is decided by whether you need long memory of sparse gradients or a bounded, adaptive step.

EXAMINER'S CHECKLIST

- Writes all three update rules with the accumulator defined.
- Explains the stall of AdaGrad and the effective memory of RMSProp.
- Derives the need for Adam's bias correction.
- Compares memory, hyper-parameters and regime of use.
- Commits to a recommendation with a reason.

KEYWORDS TO WRITE IN THE EXAM
adaptive learning rate · accumulator · exponential moving average · effective memory · bias correction · sparse gradients · non-stationary objective · momentum · generalisation

Q8 Discuss the trade-offs between early stopping, dropout and data augmentation

CO3 · Analyse–Evaluate · 10 marks · ~3 pages

Three regularisers at three levels

All three fight overfitting, but they act on different objects, and that is what makes the trade-offs examinable:

Operates on	Mechanism	
early stopping	the number of optimisation steps	restricts the distance travelled from the initialisation
dropout	the activations	injects multiplicative noise, trains an ensemble
augmentation	the training data	enlarges the support of the empirical distribution

Early stopping

Train, monitor the validation loss each epoch, and return the parameters from the epoch with the lowest validation loss. Its theoretical justification is a Taylor expansion of the training loss around the initialisation: for a quadratic objective, stopping after a number of steps is equivalent to a ridge penalty with

$$\lambda \approx \frac{1}{\tau \eta} \tag{3.33}$$

where τ is the number of steps and η the step size, so limiting the number of steps limits how far the weights can grow — early stopping *is* L_2 regularisation with a step-count hyper-parameter. Its cost is that it needs a validation split, and that split is data removed from training. It is otherwise free: no change to the model, no extra compute at inference, no extra hyper-parameter beyond the patience.

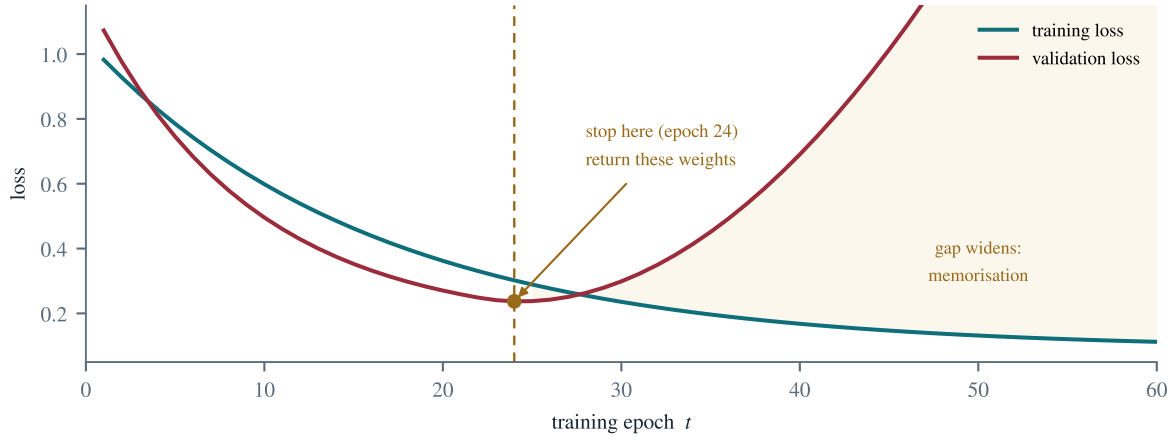


Fig. 3.6 Validation loss bottoming out; the weights at that epoch are the ones to keep. Everything after the dotted line is memorisation.

Dropout

Discussed in detail earlier; for the trade-off what matters is that dropout is a *training-time* intervention with a *training-time* cost: two to three times more epochs, a noisier loss curve, and a need to raise the learning rate and lengthen the schedule. At inference it costs nothing. It is most effective when the overfitting is driven by co-adaptation among many dense units, and least effective in convolutional stacks, where weight sharing already limits capacity.

Data augmentation

Generate new labelled examples by applying transformations that leave the label invariant: crops, flips, rotations, colour jitter, cutout; for audio, time shifts and masking; for text, synonym replacement and back-translation. Augmentation changes

the *effective* dataset size, which is the only intervention with a guarantee that variance falls as the data grows. Its cost is domain knowledge — choosing a transform that is genuinely label-preserving — plus compute per epoch, and a risk: an aggressive transform (large rotations on digit recognition, horizontal flip on character recognition) teaches the model an invariance the task does not have, converting variance into bias.

Head-to-head trade-offs

Early stopping	Dropout	Augmentation	
extra data needed	validation split	none	none, but domain knowledge
training cost	none	2–3× epochs	compute per epoch
inference cost	none	none (inverted)	none
main risk	noisy validation → wrong epoch	underfitting if the rate is too high	label-destroying transform
interacts with	learning-rate schedule	batch norm, weight decay	batch norm statistics
best when	any model, always safe	wide dense layers	images, audio, text
theoretical link	L_2 with a step-count coefficient	ensemble of 2^n nets	enlarged empirical distribution

Combining them

The three are complementary and are used together, but not blindly:

- Choose augmentation first — it is the only one that adds information rather than restricting the model.
- Add early stopping always, because it is free and protects against the tail of the curve.
- Add dropout only where capacity is concentrated: the classifier head, wide fully connected blocks. Skip it in convolutional trunks that already use batch normalisation.
- Re-tune the schedule after adding any of them: all three lengthen the time to convergence, and a schedule tuned without dropout will stop training too early with it.

WHAT THE EXAM IS REALLY TESTING

Not the definitions — those are one line each — but whether you can say *what each one costs and what it can break*. A candidate who writes "use all three for best accuracy" without mentioning the validation split, the epoch multiplier, or the label-destroying transform has not answered the question.

GOLDEN LINE

Early stopping limits how far the weights travel, dropout averages over sub-networks at training time, and augmentation enlarges the data; the first is free but needs a validation split, the second costs epochs, the third costs compute and domain knowledge — and only the third adds information.

EXAMINER'S CHECKLIST

- Assigns each method to the level it acts on: steps, activations, data.
- Derives the early-stopping relation to the L_2 coefficient.
- Quantifies dropout's cost as training epochs, not arithmetic.
- Names the failure mode of an aggressive augmentation.
- Gives an order for combining them and says why.

KEYWORDS TO WRITE IN THE EXAM

early stopping · patience · effective L_2 · dropout rate · data augmentation · label-preserving transform · effective dataset size · co-adaptation · convergence time

UNIT IV

Convolutional Neural Networks

Layer design, output geometry, parameter sharing and the prior that makes vision learnable

UNIT AT A GLANCE

#	Question	Open with
1	Designing a CNN: layers, kernels, strides, pooling	Choose the geometry first — kernel, stride, padding — then let the numbers fall out of the output formula.
2	How kernel, stride and padding set the output size and the cost	The output formula and the FLOP count; always give the worked numbers.
3	Fully connected versus convolutional layers	Local connectivity and shared weights turn a quadratic parameter bill into a constant one.
4	Convolution and pooling as architectural priors	They encode translation equivariance and local tolerance — powerful when true, harmful when false.
5	Pooling when the exact location matters	Max-pooling throws away position; use stride-1 convolutions, dilated kernels or attention when coordinates are the answer.
6	Flatten versus global average pooling in a multiclass CNN	GAP removes the position-dependent dense block and the parameter explosion that comes with it.
7	Underfitting caused by pooling, and what to do instead	Aggressive pooling destroys the evidence before the classifier sees it.
8	Justifying a CNN architecture for a real application	Derive the design from the data: resolution, invariance, class count, compute budget.

Q1 Design a CNN for an image classification task — layers, kernel sizes, strides and pooling

CO4 · Apply–Create · 10 marks · ~3 pages

The design problem

A convolutional network is a sequence of stages. Each stage applies convolutions that **shrink the spatial grid and grow the channel count**, and the network ends with a classifier head that maps a feature vector to class scores. Designing the network means choosing five numbers per stage: the kernel size K, the number of filters C_{out}, the stride S, the padding P, and whether a pooling layer follows. Every choice has a consequence that must be stated.

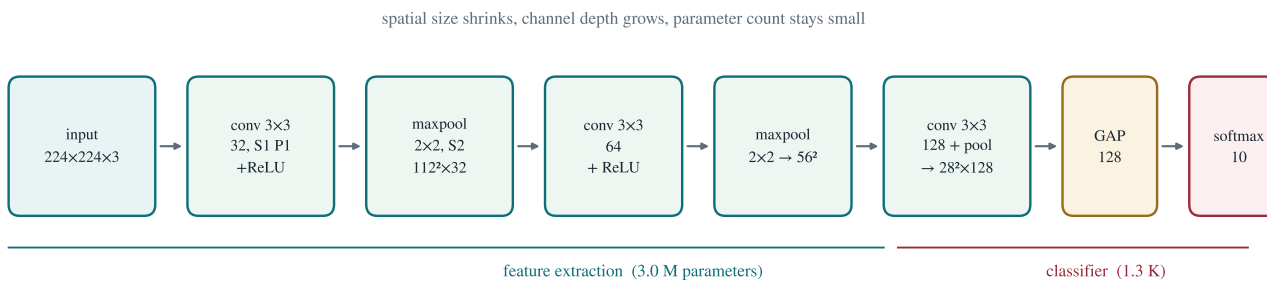


Fig. 4.1 A canonical design: three convolution stages with pooling, then global average pooling and a softmax head.

Layer by layer

Convolutional layer

A filter of size $K \times K \times C_{in}$ slides over the input; at every position it computes a dot product plus a bias, producing one value of one output channel. With C_{out} filters the layer learns C_{out} different detectors. The output is

$$y_{i,j,c} = \sum_{u=0}^{K-1} \sum_{v=0}^{K-1} \sum_{d=0}^{C_{in}-1} w_{u,v,d,c}\, x_{i+u,\, j+v,\, d} + b_c \tag{4.1}$$

Key design points:

- **Kernel size.** A 3×3 kernel is the default: two stacked 3×3 convolutions see a 5×5 region with $2 \cdot 9 = 18$ weights per channel pair instead of 25, so depth buys receptive field more cheaply than width. 1×1 convolutions mix channels and are used to change depth; 5×5 or 7×7 appear only in the first layer, where the input has three channels and a large kernel is affordable.
- **Number of filters.** Grow geometrically with depth ($32 \to 64 \to 128$) while the spatial size halves, so that the total activation volume — and therefore the memory — stays roughly constant.
- **Stride.** A stride of two halves the resolution inside the convolution itself and is the modern replacement for a pooling layer; a stride of one preserves resolution.
- **Padding.** Padding of about half the kernel with a unit stride keeps the size unchanged ("same"), no padding shrinks it ("valid").

Activation

ReLU, applied elementwise: cheap, and it does not saturate for positive inputs, which keeps gradients alive in deep stacks. Its known failure is the dead unit; leaky ReLU or ELU is the fix when it appears.

Pooling

Max-pooling with a 2×2 window and stride two halves both dimensions and keeps the strongest response in each window, giving small-shift tolerance. Average pooling is used instead at the very end of some architectures; global average pooling collapses each channel to one number.

Normalisation

Batch normalisation after the convolution and before the activation stabilises the distribution of pre-activations, allows a learning rate roughly ten times larger, and acts as a mild regulariser.

Classifier head

Global average pooling, then one dense layer of width equal to the number of classes, then softmax. Dropout is optional; with global pooling there are very few parameters left to overfit.

A complete specification

Stage	Layer	Kernel	Stride/pad	Output	Parameters
0	input	—	—	$224^2 \times 3$	0
1	conv + BN + ReLU	3×3, 32	1 / same	$224^2 \times 32$	896
2	max-pool	2×2	2 / valid	$112^2 \times 32$	0
3	conv + BN + ReLU	3×3, 64	1 / same	$112^2 \times 64$	18,496
4	max-pool	2×2	2 / valid	$56^2 \times 64$	0
5	conv + BN + ReLU	3×3, 128	1 / same	$56^2 \times 128$	73,856
6	max-pool	2×2	2 / valid	$28^2 \times 128$	0
7	global average pool	—	—	128	0
8	dense + softmax	—	—	10	1,290

The parameter count of a convolution is

$$\#\text{params} = K^2\, C_{in}\, C_{out} + C_{out} \tag{4.2}$$

so stage 3 costs $9 \cdot 32 \cdot 64 + 64 = 18{,}496$ — independent of the image size, which is the whole point of the architecture.

The rules that keep a design sane

1. Start from the data: object size in pixels fixes the receptive field, and the receptive field fixes the depth.
2. Keep the output volume (height × width × channels) roughly constant across stages; halve the spatial size when you double the channel count.
3. Prefer 3×3 kernels and stack them; use stride or pooling for the downsampling, not both at once.
4. Check every stage with the output-size formula on paper before writing code.
5. End with global average pooling rather than flatten; the parameter saving is a factor of tens.

RECEPTIVE FIELD ARITHMETIC

The receptive field after l layers grows as the kernel size plus the accumulated product of the strides: $r_l = r_{l-1} + (K_l - 1)\Pi_{i<l} S_i$ with $r_0 = 1$. Two 3×3 convolutions give 5×5; three give 7×7. A design that never exceeds the object size in pixels cannot see the object as a whole.

GOLDEN LINE

Design a CNN outward from the data: the object size sets the receptive field, the receptive field sets the depth, and the compute budget sets the channel widths — with the output-size formula checked at every stage.

EXAMINER'S CHECKLIST

- Lists the layer types and gives a concrete kernel, stride and padding for each.
- Justifies 3×3 kernels by the parameter arithmetic of stacked filters.
- Uses the output-size formula and the parameter formula.
- Explains the shrink-space / grow-channel pattern.
- Ends with global pooling plus softmax and says why.

KEYWORDS TO WRITE IN THE EXAM

convolution · kernel · stride · padding · pooling · receptive field · channel depth · batch normalisation · global average pooling · softmax head

Q2 Derive the effect of kernel size, stride and padding on output size and computational cost, with a worked example

CO4 · Apply–Analyse · 10 marks · ~3 pages

Counting the positions

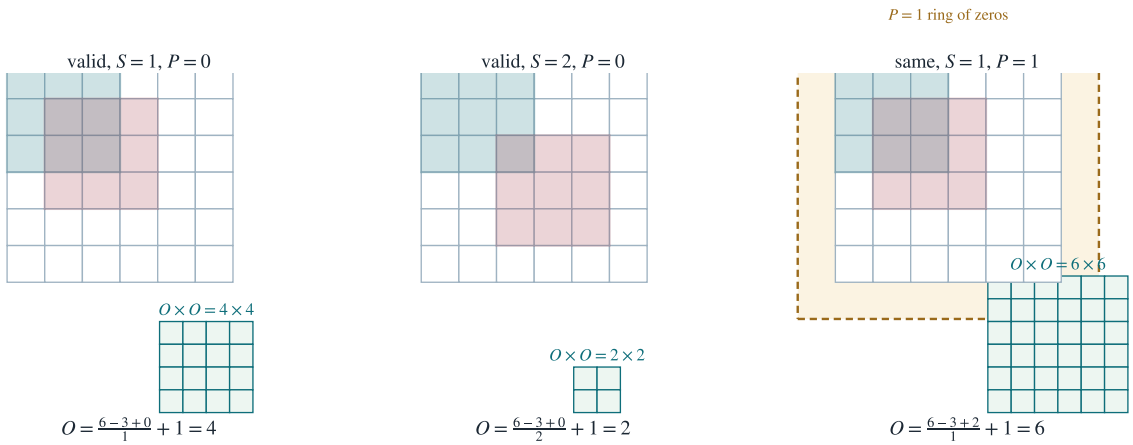
A kernel of width K placed on an input of width W with padding P on both sides moves in steps of S. The number of positions at which the kernel fits completely is the number of integers $i \geq 0$ with $iS \leq W - K + 2P$, so that

$$O = \left\lfloor \frac{W - K + 2P}{S} \right\rfloor + 1 \tag{4.3}$$

The floor is essential: a kernel that would hang over the edge is simply not placed. The same formula applies independently to height and width, and it is the single most useful line in the whole unit.

The three regimes

Regime	Setting	Output size	Typical use
valid	no padding	smaller than the input	shrinking at every layer
same	half-kernel padding, unit stride	equal to the input	keeping resolution while adding depth
full	maximum padding	larger than the input	transposed convolution, upsampling
strided	stride above one	about the input over the stride	learned downsampling
dilated	spacing between taps	effective kernel grows with spacing	wide context, no resolution loss



same 6×6 input and 3×3 kernel: stride and padding decide the output size

Fig. 4.2 The same 6×6 input with a 3×3 kernel: valid stride 1 gives 4×4, valid stride 2 gives 2×2, and padding 1 with stride 1 keeps 6×6.

Worked example

Take an input of $224 \times 224 \times 3$ and a convolutional layer with $K = 3$, $S = 1$, $P = 1$ and $C_{out} = 64$.

1. **Output size.** $O = (224 - 3 + 2)/1 + 1 = 224$, so the map is $224 \times 224 \times 64$ — a "same" convolution.
2. **Parameters.** $K^2 C_{in} C_{out} + C_{out} = 9 \cdot 3 \cdot 64 + 64 = 1{,}792$.
3. **Multiply–accumulate operations.** Each output element needs $K^2 C_{in} = 27$ such operations, so

$$\text{MACs} = H_{out} W_{out} C_{out} K^2 C_{in} = 224 \cdot 224 \cdot 64 \cdot 27 = 86.7 \times 10^6 \tag{4.4}$$

1. **Floating-point operations.** One multiply–accumulate is one multiply and one add, so

$$\text{FLOPs} = 2\, HWK^2 C_{in} C_{out} = 173.4 \times 10^6 \tag{4.5}$$

1. **Effect of stride.** With a stride of two the output becomes $112 \times 112 \times 64$ and the cost falls by a factor of four — downsampling early is the cheapest optimisation available.
2. **Effect of doubling the kernel.** A 5×5 kernel at the same output size multiplies both the parameters and the cost by $25/9 \approx 2.8$.

The general cost model

For a whole network the cost is the sum over layers of

$$C_l = 2\, H_l W_l K_l^2 C_l^{in} C_l^{out} \tag{4.6}$$

and the memory is dominated by the activations that must be stored for the backward pass, $H_l W_l C_l$ per layer per example. Two consequences worth stating:

- Cost is **linear in the number of pixels and quadratic in the kernel width**, so stacks of small kernels are strictly cheaper than one wide kernel for the same receptive field.
- Cost is **linear in both channel counts**, so the first layer — three input channels — is cheap, while the last convolutional layer with many channels is expensive even though its map is small.

WHY THIS MATTERS IN AN EXAM

Numbers separate a real answer from a recitation. If you can say "a 3×3 convolution on a $224 \times 224 \times 3$ input with 64 filters costs 87 million multiply–accumulates and 1,792 weights", you have demonstrated that you understand both the geometry and the parameter sharing.

GOLDEN LINE

Output size is the floor of the padded width minus the kernel over the stride, plus one; cost is twice the product of output size, kernel area and both channel counts — stride is the cheap way to cut cost, kernel width is the expensive one, and the parameter count does not depend on the image size at all.

EXAMINER'S CHECKLIST

- Derives the output formula from the count of valid kernel positions, floor included.
- Distinguishes valid, same, full, strided and dilated.
- Works one full numeric example: output size, parameters, operations, FLOPs.
- Shows the effect of changing stride and changing kernel size.
- States the general cost model and the memory term.

KEYWORDS TO WRITE IN THE EXAM

output size formula · valid padding · same padding · stride · dilation · receptive field · multiply–accumulate · FLOPs · parameter count · activation memory

Q3 Compare fully connected and convolutional layers in terms of parameter sharing, local connectivity and scalability

CO4 · Understand–Analyse · 10 marks · ~2.5 pages

The parameter arithmetic

Consider an input image of $256 \times 256 \times 3$ and a hidden layer of 1,000 units.

- **Fully connected:** every input element connects to every unit, so the weight matrix is $196{,}608 \times 1{,}000$ — about 196 million parameters in one layer.
- **Convolutional:** a bank of 64 filters of size $3 \times 3 \times 3$ has $9 \cdot 3 \cdot 64 = 1{,}728$ weights, and the layer still produces $256^2 \times 64$ activations.

The ratio is five orders of magnitude, and it comes entirely from two structural choices.

Local connectivity

A convolutional unit sees only a $K \times K$ neighbourhood, so

$$y_{i,j} = \sum_{u,v} w_{u,v}\, x_{i+u,j+v} + b \tag{4.7}$$

instead of a sum over the whole input. The assumption is that nearby pixels are the ones that matter for a local feature — an edge, a corner, a colour transition. The receptive field grows with depth, so locality at each layer does not limit the global context the network can eventually represent: three 3×3 layers already see 7×7.

Weight sharing

The same $K^2 C_{in}$ weights are reused at every spatial position. Equivalently, a convolution is a fully connected layer in which most weights are tied to each other: the weight from input position offset by (u, v) to an output position is the same for every output position. Three consequences follow:

1. **Parameter count is independent of the image size.** Doubling the resolution does not change the number of weights at all; it only changes the number of activations.
2. **Translation equivariance is built in.** If the input shifts by one pixel, the output shifts by one pixel, because the same filter is applied everywhere.
3. **Statistical strength.** Every position provides a training example for the same filter, so a filter is learned from millions of patches rather than from one.

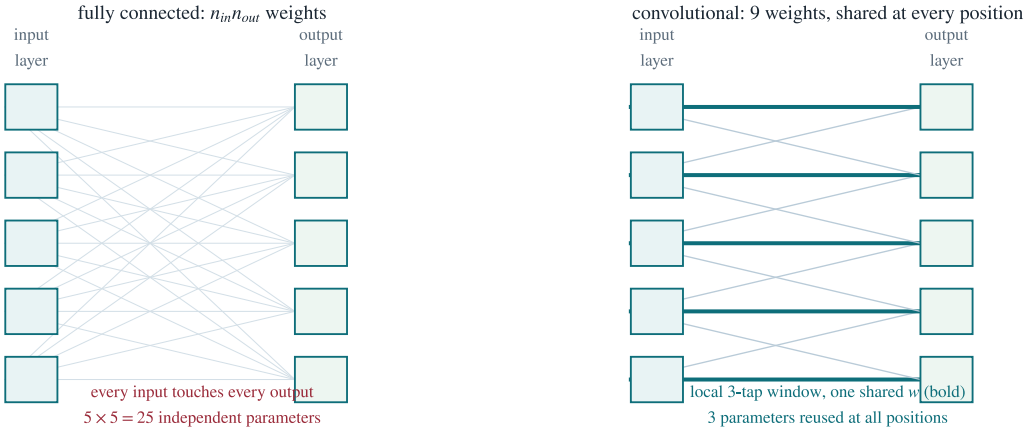


Fig. 4.3 Left: a fully connected layer, one weight per input–output pair. Right: a convolutional layer, a few weights shared across positions.

Scalability

Property	Fully connected	Convolutional
parameters	grow with the square of the input size	independent of resolution
memory for weights	can exceed the device memory	a few kilobytes per filter
inductive bias	none; must learn locality	locality and translation equivariance
data efficiency	needs many examples per pixel	exploits every patch
sensitivity to a shift	the input vector changes entirely	the output shifts, the values do not
best used for	tabular data, final classifier	images, spectrograms, sequences

What is lost

The two priors are assumptions, and they are wrong when the data does not respect them:

- **Global relationships** are not modelled by a small kernel; distant parts of the image interact only after many layers, or never.
- **Arbitrary orientation** is not encoded — a filter trained on upright digits will not fire on rotated ones unless augmentation teaches it.
- **Exact coordinates** are progressively destroyed by strided convolutions and pooling, which is fatal for tasks such as keypoint localisation.

A modern network therefore mixes the two: convolutional trunks for the local structure, and a small fully connected or attention head for the global decision.

THE ONE-SENTENCE ANSWER

A fully connected layer treats every input position as an independent feature, so its parameter count explodes with resolution and it must learn from scratch that nearby pixels are related; a convolutional layer hard-codes locality and shares weights across positions, which cuts the parameters by orders of magnitude and buys translation equivariance for free.

GOLDEN LINE

Local connectivity says "look here", weight sharing says "look for the same thing everywhere" — together they reduce the parameter bill from quadratic in the input size to a constant, and make the model independent of image size.

EXAMINER'S CHECKLIST

- Puts real numbers on the parameter counts.
- Defines local connectivity and weight sharing formally.
- Shows the parameter count is independent of resolution.
- States equivariance, and why it is equivariance rather than invariance.
- Names the situations where the priors fail.

KEYWORDS TO WRITE IN THE EXAM

local receptive field · weight sharing · parameter tying · translation equivariance · inductive bias · data efficiency · scalability · resolution independence

Q4 Discuss convolution and pooling as architectural priors — their benefits and their limitations

CO4 · Analyse–Evaluate · 10 marks · ~2.5 pages

What a prior is

A prior is a constraint on the hypothesis space that is imposed before any data is seen. Convolution and pooling impose two:

1. **Locality and stationarity:** the same feature detector is useful at every position, and it only needs a small neighbourhood. This is the prior of *translation equivariance*.
2. **Local tolerance:** small shifts, and small deformations, should not change the answer much. This is the prior of *approximate invariance*, implemented by pooling and stride.

Inserting a correct prior reduces the number of examples needed to learn; inserting a wrong one caps the achievable accuracy.

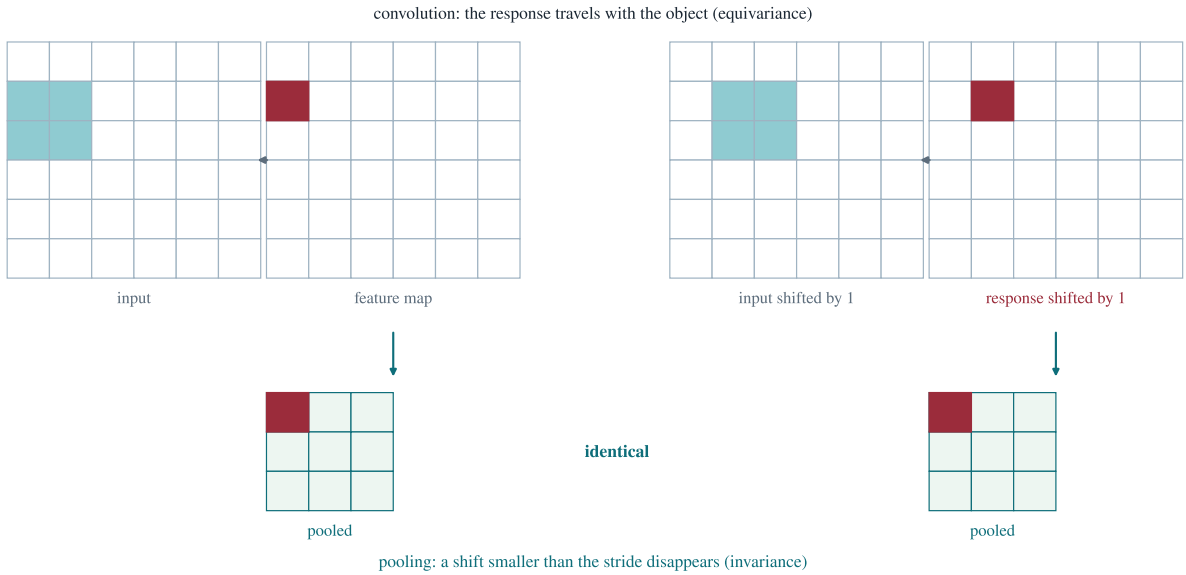


Fig. 4.4 Top: the same filter responds at the new position when the object moves. Bottom: after 2×2 pooling a shift of one cell disappears, so the pooled map is identical.

Benefits

- **Sample efficiency.** Because each filter is trained by every position, a convolutional model learns from far fewer labelled images than a dense network with the same expressive power.
- **Parameter efficiency.** A factor of 10^4–10^5 in weights, which translates directly into memory, speed and the ability to run on small devices.
- **Equivariance is exact.** For a convolution without padding artefacts, shifting the input shifts the feature map exactly. Pooling, applied after, coarsens this into tolerance: a shift smaller than the pooling stride leaves the pooled map unchanged.
- **Compositionality.** Stacking builds a hierarchy — edges, then textures, then parts, then objects — which matches the statistics of natural images and is what makes deep CNNs interpretable layer by layer.

Limitations

Prior	Assumption	Where it fails
locality	relevant evidence is nearby	global context: scene classification, long-range dependencies
weight sharing	the same detector is useful everywhere	position-dependent classes: "sky above, road below"
translation equivariance	a shifted object is the same object	tasks needing coordinates: detection offsets, keypoints
pooling invariance	small shifts are irrelevant	fine-grained recognition, counting, localisation
fixed grid	data lie on a regular lattice	point clouds, graphs, meshes
fixed kernel	scale is known in advance	objects at wildly different scales without pyramids

Each row is a real failure mode with a known remedy: global context via attention or dilated convolutions; position dependence via positional channels or by not sharing the last layers; localisation via stride-1 features and skip connections; scale via feature pyramids or multi-scale training; irregular domains via graph convolution.

The cost of removing a prior

Capsule networks, vision transformers and dense networks with learned position embeddings all abandon one or both priors, and all pay for it: they need far more data or much stronger augmentation, and they are usually heavier to train. The empirical lesson of the last decade is that a prior is a loan against data — cheap when the assumption holds, expensive when it does not.

TWO CLAIMS THAT LOSE MARKS

"CNN is translation invariant." It is *equivariant*; invariance only appears approximately after pooling or global averaging, and only for shifts smaller than the pooling window.

"Pooling gives invariance to rotation and scale." It does not; it gives tolerance to small translations. Rotation and scale invariance come from augmentation, multi-scale features, or equivariant architectures.

GOLDEN LINE

Convolution builds in locality and weight sharing (equivariance), pooling builds in tolerance to small shifts (approximate invariance); both are loans against data that pay off enormously on natural images and default badly on coordinates, global context or irregular domains.

EXAMINER'S CHECKLIST

- Defines a prior and names the two priors precisely.
- Gives benefits with mechanisms, not adjectives: sample efficiency, parameters, exact equivariance, hierarchy.
- Gives at least four limitations with the remedy for each.
- Distinguishes equivariance from invariance.
- Notes the data cost of abandoning the prior.

KEYWORDS TO WRITE IN THE EXAM
inductive bias · translation equivariance · approximate invariance · locality · stationarity · pooling · sample efficiency · receptive field · attention as a global prior

Q5 A task needs the exact location of an object — discuss pooling and propose a better strategy

CO4 · Analyse–Create · 10 marks · ~2.5 pages

What pooling does to position

Max-pooling with window k and stride s maps each window to a single number. Every position inside a window produces the same output, so the map has lost all information finer than the stride — formally, the pooled representation is invariant to translations by less than the stride. For classification that is exactly what we want: "there is a tumour in this scan" should not depend on a two-pixel shift. For localisation it is a disaster: the answer *is* the coordinate.

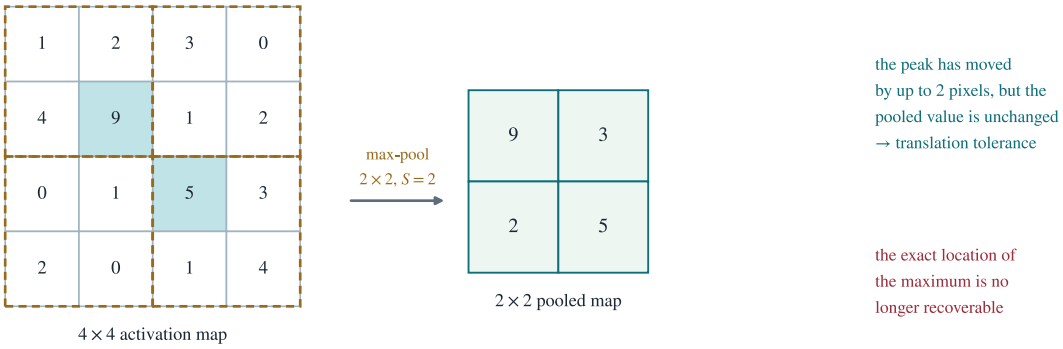


Fig. 4.5 Max-pooling halves the grid and keeps the strongest response; the peak survives in value but its position is quantised to the block.

Quantitatively, if a network has four 2×2 pooling stages, one output cell corresponds to 16×16 input pixels. Recovering the position to one pixel from such a feature map is impossible without extra machinery; the best a naive decoder can do is half a cell in each direction.

Strategies that keep position

1. **Reduce or remove pooling.** Use stride-1 convolutions throughout the trunk and down-sample only as much as the compute budget forces. Cost rises, but the feature map keeps its resolution.
2. **Dilated (atrous) convolutions.** Insert zeros between kernel taps so the receptive field grows without any reduction in resolution; the effective kernel grows with the spacing. A stack with spacings 1, 2, 4, 8 sees a wide context while every layer stays at full resolution — the design used in semantic segmentation.
3. **Encoder–decoder with skip connections.** Pool in the encoder to get context, then upsample and concatenate the high-resolution features from the encoder. The decoder regains the detail that the encoder discarded, and the skip connections carry the coordinates.
4. **Transposed convolution instead of unpooling.** Learn the upsampling instead of repeating values; it is the standard decoder operation.
5. **Predict offsets, not masks.** Regress the box or the keypoint coordinates directly from a coarse feature map with a small head, using an offset term so that the sub-cell position is representable.
6. **Soft-argmax instead of max.** Replace the hard maximum by a weighted average over positions using a softmax of the activations, which is differentiable and returns a sub-pixel coordinate.

7. **Attention or deformable sampling.** Let the network sample features at learned offsets, so the sampling grid follows the object rather than the lattice.

Choosing between them

Requirement	Strategy	Cost
coarse localisation (bounding box)	keep the last pooling, add a regression head	small head
pixel-accurate segmentation	skip connections, U-Net decoder	decoder parameters
full-resolution, wide context	dilated convolutions	compute at full resolution
sub-pixel keypoints	soft-argmax or offset regression	tiny head
variable object scale	feature pyramid over several strides	memory

FRAMING THE ANSWER

Start from the trade-off in one line: *pooling buys invariance by destroying position, so any task whose label depends on position must limit it.* Then give the alternatives in order of intrusiveness — remove, replace, or recover. That structure is what earns the marks, not the list itself.

GOLDEN LINE

Pooling is an invariance operator, so it is incompatible with a task whose answer is a coordinate: keep resolution with stride-1 or dilated convolutions, restore it with skip connections, or predict the offset explicitly with a sub-pixel head.

EXAMINER'S CHECKLIST

- Explains the quantisation of position caused by pooling, with the stride-sized figure.
- Names at least four alternatives with the mechanism of each.
- Gives the effective-kernel formula for dilation.
- Distinguishes classification, detection and segmentation requirements.
- Ends with a recommendation tied to the task.

KEYWORDS TO WRITE IN THE EXAM

localisation · invariance–equivariance trade-off · dilated convolution · atrous · skip connection · U-Net · transposed convolution · soft-argmax · offset regression · feature pyramid

Q6 Describe the multiclass CNN pipeline and compare flatten with global average pooling

CO4 · Understand–Apply · 10 marks · ~2.5 pages

The pipeline end to end

For a C-class problem the network computes a score vector and turns it into a distribution:

$$z = W\,g(\cdot) + b, \qquad \hat{y}_c = \frac{e^{z_c}}{\sum_{k=1}^{C} e^{z_k}} \tag{4.8}$$

and is trained by minimising the categorical cross-entropy

$$L = -\sum_{c=1}^{C} y_c \log \hat{y}_c \tag{4.9}$$

Because the target is one-hot this reduces to the negative log-probability of the true class. The gradient has the very clean form

$$\frac{\partial L}{\partial z_c} = \hat{y}_c - y_c \tag{4.10}$$

which is why softmax with cross-entropy is the default head: the error signal is exactly the difference between predicted and true probability.

The full pipeline, in order:

1. Input normalisation (mean and standard deviation per channel).
2. Convolutional stages with batch normalisation and ReLU, alternating with downsampling.
3. An aggregation step that turns a height × width × channel tensor into a vector.
4. One dense layer to as many logits as there are classes.
5. Softmax, cross-entropy, backpropagation, and a mini-batch optimiser with a decaying learning rate.

The aggregation step: flatten

Flatten reshapes the tensor into a vector of length height × width × channels and feeds it to a dense layer of C units, giving the product of those two numbers as weights. For a $7 \times 7 \times 512$ map and 10 classes that is 250,880 weights in the last layer alone — more than an entire small network.

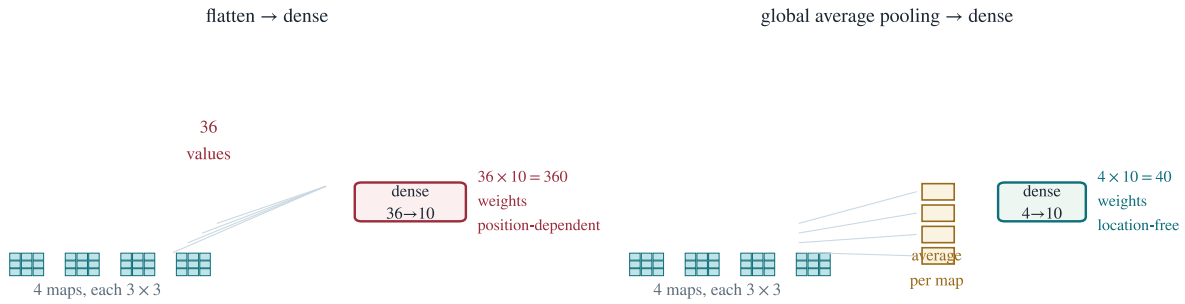


Fig. 4.6 Flatten passes every position to the classifier; global average pooling passes one number per channel.

The aggregation step: global average pooling

Global average pooling averages each channel over its spatial extent:

$$g_c = \frac{1}{HW} \sum_{i=1}^{H} \sum_{j=1}^{W} A_{i,j,c} \tag{4.11}$$

producing a C-dimensional vector, so the classifier has only the product of the channel count and the class count as weights — 5,120 in the example above, a factor of 49 fewer.

Comparison

Property	Flatten	Global average pooling
input to the head	height × width × channels	channels only
head parameters	large, grows with resolution	small, independent of resolution
position sensitivity	keeps it; can memorise layouts	discards it; treats evidence as a bag
overfitting risk	high, especially on small datasets	low
input-size flexibility	fixed; the dense layer pins the resolution	accepts any resolution
interpretability	none	each channel becomes a class-evidence map
failure mode	memorises background-position correlations	cannot localise or count

Why global pooling usually wins

Two arguments beyond the parameter count. First, the dense layer after flatten can exploit *where* a feature occurred, which on small datasets is usually a spurious correlation — the classic "cow on grass" failure — so removing the position information removes a shortcut. Second, global pooling makes the channel dimension the only interface to the classifier, which turns each channel into a class-evidence detector and yields class activation maps for free:

$$M_c(i,j) = \sum_k w_{k,c} A_{i,j,k} \tag{4.12}$$

the weighted sum of the channels with the classifier's weights for class c. That is the standard weakly supervised localisation argument from Unit II.

LINK TO THE EARLIER ANSWER

In Unit II the softmax output layer was justified by the cross-entropy gradient; here the same head is kept and only the *aggregation* before it changes. Say this explicitly: the classifier is unchanged, the interface is what global pooling rewrites.

GOLDEN LINE

Flatten hands the classifier every position and pays height × width × channels weights for the privilege; global average pooling hands it one number per channel, cutting the head by a factor of the map area, removing a shortcut, and making class activation maps possible.

EXAMINER'S CHECKLIST

- Writes softmax, cross-entropy and its gradient.
- Describes the pipeline in the right order.
- Gives the parameter counts for both choices on a concrete tensor.
- Compares position sensitivity, overfitting, input-size flexibility and interpretability.
- Recommends one and justifies it.

KEYWORDS TO WRITE IN THE EXAM

softmax · cross-entropy · logits · flatten · global average pooling · parameter explosion · class activation map · input-size flexibility · shortcut learning

Q7 Pooling is suspected of causing underfitting — explain and propose alternatives

CO4 · Analyse–Create · 10 marks · ~2.5 pages

How pooling can cause underfitting

Underfitting means the training loss itself is too high: the model cannot represent the training set. Pooling contributes to this in three distinct ways, and they should be separated in the answer.

1. **Information destruction.** Max-pooling keeps one value per window. If the label depends on several of the discarded values — the count of objects, the relative position of two parts, a thin structure — that evidence is gone before the classifier sees it. No amount of training can recover information that the forward pass removed.
2. **Capacity reduction.** The number of spatial positions reaching the later layers falls by four per 2×2 pooling stage. With fewer positions the later filters see fewer distinct contexts and the effective capacity of the network drops.
3. **Gradient sparsity.** Only the argmax position of each pooling window receives gradient. In a 2×2 window three quarters of the positions get nothing, which slows learning of exactly the fine detail that underfitting concerns.

A quick diagnostic: if training accuracy is low *and* the loss curve plateaus early while validation accuracy stays close to training accuracy, the problem is capacity or information, not regularisation. Removing pooling should then raise training accuracy — if it does, the diagnosis is confirmed.

Alternatives, in order of intrusiveness

1. **Replace pooling with a strided convolution.** Downsampling is learned rather than fixed, so the network can choose to preserve whatever the task needs instead of keeping only the maximum. Costs a kernel-sized weight tensor per layer, and usually improves accuracy at equal output size.
2. **Reduce the number of pooling stages.** Keep 2×2 pooling only where the compute budget genuinely requires it; often one stage can be removed at a modest cost.
3. **Use average pooling or a blend.** Average pooling propagates all values, though diluted; intermediate norms interpolate between summing and taking the maximum.
4. **Dilated convolutions for context without downsampling.** The receptive field grows with the spacing of the taps while the resolution stays full.
5. **Attention pooling instead of fixed pooling.** Learn a weight per position and take a weighted average, so the network decides what to keep:

$$g_c = \sum_{i,j} \alpha_{i,j,c} A_{i,j,c}, \qquad \sum_{i,j} \alpha_{i,j,c} = 1 \tag{4.13}$$

1. **Increase capacity elsewhere** if the diagnosis is capacity rather than information: more channels, a deeper trunk, a higher input resolution. This is the remedy only after the pooling question is settled, because it addresses a different cause.

What not to do

Reducing the model, adding dropout or adding weight decay are the reflexes for overfitting; applied to an underfitting model they make both curves worse. Similarly, replacing max-pooling by average pooling everywhere is rarely the answer — average pooling blurs, which on fine-grained tasks loses more than it gains.

Remedy	Fixes	Cost
strided convolution	learned downsampling	weights per layer
fewer pooling stages	information loss	compute, memory
dilated convolution	context without loss	compute at full resolution
attention pooling	fixed, task-agnostic aggregation	small attention head
more channels or depth	genuine capacity shortfall	parameters

THE EXAM TRAP

The question says *underfitting*, which is the opposite of the usual pooling complaint. An answer that talks about overfitting, dropout and early stopping is answering a different question. Keep the diagnosis explicit: high training error, small train–validation gap, plateau long before convergence.

GOLDEN LINE

Pooling causes underfitting by deleting the evidence the label depends on and by starving most positions of gradient; replace it with strided or dilated convolutions, fewer pooling stages or learned attention pooling rather than by adding capacity blindly.

EXAMINER'S CHECKLIST

- Defines underfitting by the training loss, not the test loss.
- Gives the three mechanisms: information, capacity, gradient sparsity.
- Proposes at least four alternatives with costs.
- Warns against the overfitting reflexes.
- Suggests a diagnostic that distinguishes information loss from a capacity shortfall.

KEYWORDS TO WRITE IN THE EXAM

underfitting · information loss · strided convolution · dilated convolution · average pooling · attention pooling · gradient sparsity · capacity · receptive field

Q8 Justify a CNN architecture for a real application, with every design decision tied to the data

CO4 · Create–Evaluate · 10 marks · ~3 pages

The application

Take automated grading of mangoes on a conveyor: each fruit is photographed and must be sorted into one of five quality grades. The data are 640×480 colour images, about 12,000 labelled examples, imbalanced (the top grade is rare), and the model must run on an embedded device at 30 fruits per second. Every choice below is justified by one of these four facts: **the object is large and centred, the classes differ by texture and colour, the dataset is small and imbalanced, and the compute budget is tight.**

Step 1 — Input resolution

The fruit occupies roughly 400×400 pixels, and the defects that decide the grade are millimetre-scale. Down-sampling to 128^2 would erase them; keeping the full frame would multiply the cost for no gain. A crop to 320×320 around the detected fruit is the compromise: it keeps the defect scale and halves the cost relative to the full frame. **Decision: a 320×320 centre crop, with the crop box coming from a classical colour threshold rather than a second network**, because a learned detector would double the latency budget.

Step 2 — Stem and trunk

The first layer uses a 5×5 convolution with stride two and 32 filters: a larger kernel and a stride at the input are the cheapest way to reduce resolution while capturing colour gradients, and the input has only three channels so the cost is trivial. The trunk then alternates 3×3 convolutions at 64, 128 and 256 channels with 2×2 max-pooling between stages. Depth is limited to 12 convolutional layers because the receptive field after that already exceeds the fruit, and additional depth would overfit 12,000 images.

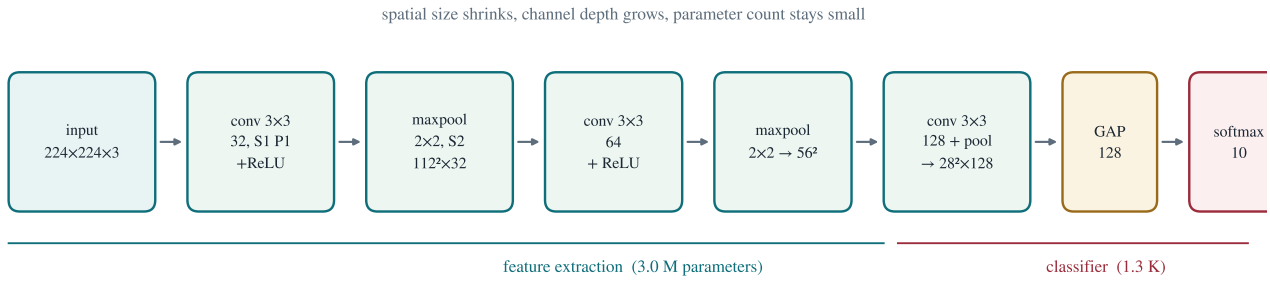


Fig. 4.7 The specification: three convolutional stages, global average pooling, and a five-way softmax with about 1,300 parameters in the head.

Step 3 — Head

Global average pooling, then a single dense layer of five units. Flatten would add tens of thousands of weights to a head that has about 1,300 with global pooling, and with 12,000 images that difference is exactly where overfitting lives. Global pooling also makes the model independent of the exact crop size, which matters because the detector's box jitters.

Step 4 — Training choices that follow from the data

Data fact	Consequence	Setting
12,000 images only	strong augmentation, regularisation	flips, rotations of $\pm 15°$, colour jitter, cutout
class imbalance	reweight the loss	class weights inverse to frequency, or focal loss
texture decides the grade	keep resolution late	pool only three times, not five
embedded device, 30 fps	small head, quantisation-aware training	global pooling head, INT8 post-training quantisation
crops jitter	input-size flexibility	global pooling rather than flatten

The optimiser is stochastic descent with momentum 0.9 and a cosine schedule, because the last accuracy points matter on a small dataset and it finds flatter minima than Adam here; batch normalisation is used in every block so that the learning rate can be 0.05 without divergence. Early stopping on a stratified validation split protects the rare grade from being memorised last and forgotten first.

Step 5 — What is deliberately not used

- No fully connected layers before the global pooling — they would add parameters without adding invariance.
- No depthwise separable convolutions in the trunk: they would halve the cost, but the trunk is not the bottleneck; the camera pipeline is.
- No pretrained backbone, because the domain (fruit skin texture) is far from natural-image pretraining and fine-tuning a large model would exceed the latency budget; a small model trained from scratch with heavy augmentation is the honest baseline. If accuracy were short, the next experiment would be a small mobile backbone fine-tuned at 320^2, not a deeper custom net.

Step 6 — How the design would be validated

1. Baseline: the 12-layer net above, measured on a stratified test split with per-class recall, not just overall accuracy, because the rare grade matters.
2. Ablations, one at a time: global pooling versus flatten; three pooling stages versus two; augmentation on and off.
3. Latency measured on the target device, not on the training GPU.

4. Error analysis by grade: confusion between adjacent grades is expected and tolerable; confusion between the best and the worst grade is a design failure.

WHAT "JUSTIFY" MEANS

Every sentence must connect a design choice to a property of the data or the deployment. "I used 3×3 kernels because they are standard" earns nothing; "I used 3×3 kernels because the defects are two to five pixels wide and a 7×7 kernel would smear them" earns the mark.

GOLDEN LINE

Justify an architecture by deriving it from the data — object scale fixes the resolution and receptive field, class imbalance fixes the loss, dataset size fixes the head and the augmentation, and the deployment target fixes the depth and the quantisation.

EXAMINER'S CHECKLIST

- States the application's data facts before any architecture choice.
- Ties resolution and receptive field to object and defect size.
- Chooses global pooling and gives the parameter arithmetic.
- Uses the imbalance and the deployment budget to justify loss and depth.
- Names what was deliberately excluded and why, and how the design would be validated.

KEYWORDS TO WRITE IN THE EXAM

receptive field · input resolution · global average pooling · class imbalance · augmentation · edge deployment · quantisation · ablation study · per-class recall · design justification

UNIT V

Recurrent Networks and Autoencoders

Unrolling over time, the vanishing gradient, gated memory, sequence transduction and learned representations

UNIT AT A GLANCE

#	Question	Open with
1	An unfolded simple RNN and parameter sharing over time	One cell reused at every step: the same recurrent matrix appears everywhere, which is both the point and the problem.
2	Vanishing and exploding gradients, and why the LSTM helps	The Jacobian is a product of many matrices; gates replace that product by an elementwise one.
3	Standard, bidirectional, deep RNN and LSTM compared	Four answers to 'which recurrence?', differing in direction, depth and state update.
4	LSTM gates and the cell state	Forget, input and output gates acting on a conveyor-belt state.
5	A strategy for long sequences where a simple RNN fails	Clip, truncate, gate, restructure, then attend — in that order.
6	Encoder–decoder sequence-to-sequence design and its training objective	The decoder factorises the target distribution autoregressively, conditioned on the context vector.
7	An autoencoder for dimensionality reduction	A bottleneck plus a reconstruction loss; a linear autoencoder spans the PCA subspace.
8	Autoencoders and representation learning	Denoising, sparse and contractive variants learn features that transfer.

Q1 Draw a simple RNN unfolded over time and explain parameter sharing across the time steps

CO5 · Understand–Apply · 10 marks · ~2.5 pages

The recurrence

A simple (Elman) recurrent network processes a sequence one element at a time, carrying a hidden state:

$$h_t = \tanh\left(W_{xh}x_t + W_{hh}h_{t-1} + b_h\right) \tag{5.1}$$

$$y_t = \mathrm{softmax}\left(W_{hy}h_t + b_y\right) \tag{5.2}$$

The state is a fixed-length summary of everything seen up to time t; the output is computed from that summary. The same three matrices are used at every step — that is the definition of a recurrent layer, and everything else follows from it.

Unfolding

"Unfolding" means drawing the computation graph for all T steps explicitly. The cycle from one state to the next becomes a chain of copies of the same cell, each with its own input and output, and with the weights **tied** across the copies:

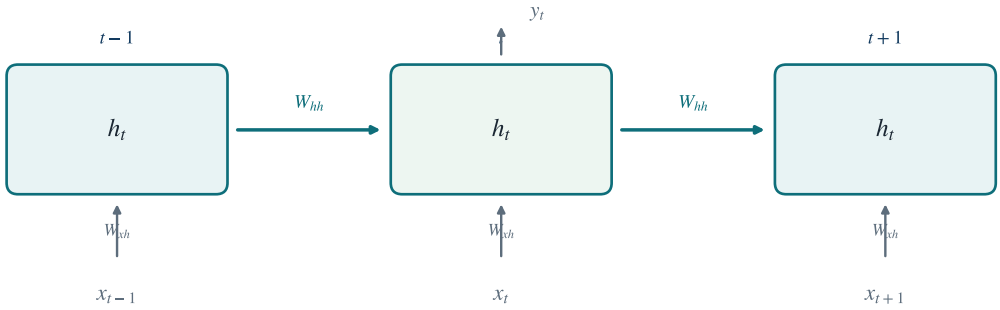


Fig. 5.1 The same cell drawn three times; the input, recurrent and output weight matrices are shared, and the state is the only thing that passes between copies.

The unfolded graph is a feedforward network of depth equal to the sequence length, whose layers happen to share parameters. This is why a recurrent network can be trained by ordinary backpropagation: there is no new algorithm, only a new graph.

Parameter sharing

Write the parameter set as the three weight matrices and two bias vectors, with sizes

$$|W_{xh}| = d_x d_h, \qquad |W_{hh}| = d_h^2, \qquad |W_{hy}| = d_h d_y \tag{5.3}$$

so the total is fixed by the hidden width and the vocabulary, and **independent of the sequence length**. Four consequences, all examinable:

1. **A variable-length input is handled by one model.** A sequence of 10 or 10,000 steps uses the same weights; there is no need to change the architecture, only the batch.
2. **Statistical strength.** Every time step is a training example for the same transition function, exactly as every image position is a training example for a shared filter. This is weight sharing in time, and it is why recurrent models generalise across lengths.
3. **Constant memory.** The model size does not grow with the sequence, unlike a dense model over a fixed window of T steps, whose parameter count grows linearly in T.
4. **The Markov assumption is replaced by a learned state.** The state is not a fixed window; it is a learned compression of the past, refreshed at every step.

Backpropagation through time

The loss sums over time, and the gradient with respect to the shared weights is the sum of the contributions from every step:

$$\frac{\partial L}{\partial W_{hh}} = \sum_{t=1}^{T} \frac{\partial L_t}{\partial W_{hh}}, \qquad \frac{\partial L_t}{\partial h_k} = \frac{\partial L_t}{\partial h_t} \prod_{i=k+1}^{t} \frac{\partial h_i}{\partial h_{i-1}} \tag{5.4}$$

$$\frac{\partial h_i}{\partial h_{i-1}} = \operatorname{diag}\left(1 - h_i^2\right) W_{hh} \tag{5.5}$$

So backpropagation through time is ordinary backpropagation on the unfolded graph, accumulating into a single gradient for the tied weights. The product in the second equation is the origin of the vanishing-gradient problem of the next answer: it is a product of many matrices, and products of matrices either shrink or blow up.

WHY THE STATE MATTERS

The hidden state plays the role of a program's memory. The difference from an n-gram model is that the state is continuous and learned, so the model can represent unbounded dependencies in principle — though in practice the gradient decay described next bounds what a simple recurrent net can learn.

GOLDEN LINE

Unfolding turns a loop into a chain of identical cells with tied weights, so the parameter count is fixed regardless of sequence length, and training is ordinary backpropagation on that chain — with gradients accumulated over all time steps.

EXAMINER'S CHECKLIST

- Draws the folded cell and the unfolded chain, labelling the shared matrices.
- Writes the state and output equations.
- States that the parameter count is independent of the length, with the sizes.
- Explains why unfolding allows ordinary backpropagation.
- Writes the gradient through time as a product of Jacobians.

KEYWORDS TO WRITE IN THE EXAM

recurrence · hidden state · unfolding · weight tying · backpropagation through time · parameter sharing in time · variable-length input · Jacobian product

Q2 Explain the vanishing and exploding gradient problem and how the LSTM mitigates it

CO5 · Understand–Analyse · 10 marks · ~3 pages

The product that causes it

From backpropagation through time, the gradient of the loss at a late step with respect to an early state is a product of Jacobians:

$$\frac{\partial L_t}{\partial h_k} = \frac{\partial L_t}{\partial h_t} \prod_{i=k+1}^{t} \operatorname{diag}\left(1 - h_i^2\right) W_{hh} \tag{5.6}$$

Taking norms and using submultiplicativity,

$$\left\| \frac{\partial L_t}{\partial h_k} \right\| \le \left\| \frac{\partial L_t}{\partial h_t} \right\| \prod_{i=k+1}^{t} \left\| \operatorname{diag}\left(1 - h_i^2\right) \right\| \|W_{hh}\| \le C\,\gamma^{\,t-k} \tag{5.7}$$

where the base γ is the product of the largest derivative of the squashing function and the spectral norm of the recurrent matrix, and the derivative of the tanh is at most one. The bound is geometric in the distance between the two steps, and that is the whole story:

- if the base is below one the gradient **vanishes** exponentially, so the network cannot learn dependencies longer than roughly the reciprocal of the decay;
- if the base is above one the gradient **explodes** exponentially, so the update overshoots and the loss becomes not-a-number.

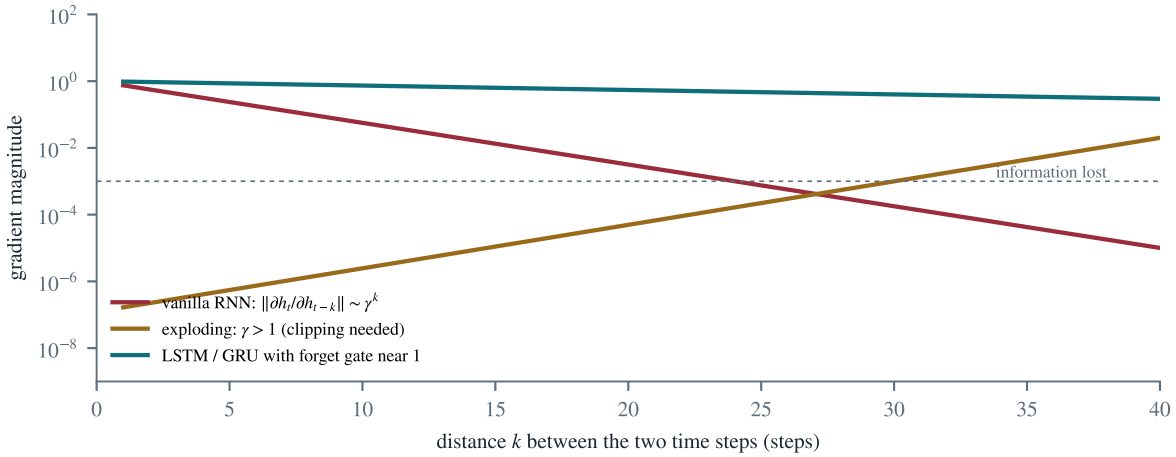


Fig. 5.2 Gradient magnitude against separation in time. The vanilla recurrence decays geometrically; the LSTM path with a forget gate near 1 is almost flat.

The vanishing case, in words

The chain rule multiplies a factor smaller than one at every step. After 50 steps a per-step factor of 0.9 leaves only about 0.005 of the original signal; after 100 steps, about 2×10^{-5}. The consequence is not merely slow learning: the parameters that would capture a long-range dependency receive no gradient at all, so the model *cannot* represent the dependency. This is why a simple recurrent network appears to work on short sequences and fails on long ones.

The exploding case

The same product with a base above one makes the gradient huge, and a single large step can destroy the weights. The standard remedy is simple and effective — **gradient clipping**:

$$g \leftarrow g \cdot \min\left(1, \frac{c}{\|g\|}\right) \tag{5.8}$$

with the threshold c typically between one and five. It bounds the step without changing its direction. Clipping fixes exploding gradients; it cannot fix vanishing ones, because scaling a tiny gradient up amplifies noise.

Why the LSTM helps

The LSTM adds a cell state with an almost additive update:

$$c_t = f_t \odot c_{t-1} + i_t \odot g_t \tag{5.9}$$

Differentiating with respect to the previous state,

$$\frac{\partial c_t}{\partial c_{t-1}} = \operatorname{diag}(f_t) + \text{terms through the gates} \tag{5.10}$$

and when the forget gate is near one, this Jacobian is close to the identity. The crucial difference from the simple recurrence is structural: the path from one cell state back to an earlier one is a product of **diagonal** matrices, not a product of the same dense matrix. A product of diagonal matrices is just an elementwise product of scalars, so

$$\frac{\partial c_t}{\partial c_k} = \prod_{i=k+1}^{t} \operatorname{diag}(f_i) \tag{5.11}$$

has no repeated mixing of directions: each cell of the state can carry its own memory with its own decay rate. A cell whose forget gate learns to sit at 0.99 has an effective memory of about a hundred steps; one at 0.5 has a memory of two steps. The network therefore *learns per-channel time constants* rather than being forced into a single global one.

Two further details complete the answer:

1. **The forget gate bias is initialised to one** (or a positive constant), so that at the start of training the gates are open and the gradient flows; the model then learns to close them.

2. **The gates are sigmoids**, so the forget values lie between zero and one and the product cannot explode by itself; exploding gradients in an LSTM come from the input path, which is why clipping is still used.

THE PRECISE CLAIM

The LSTM does not abolish the product of Jacobians; it replaces a product of dense mixing matrices by a product of diagonal gates, and it lets the network choose gate values near one. That is the sentence that distinguishes a correct answer from a slogan.

GOLDEN LINE

A dependency across many steps passes through as many Jacobians, so the gradient scales geometrically; the LSTM replaces the repeated dense product by a diagonal gate product on an additive cell-state path, giving each channel its own learnable time constant, and clipping handles the exploding case.

EXAMINER'S CHECKLIST

- Writes the Jacobian product and derives the geometric bound.
- Distinguishes vanishing (cannot learn) from exploding (unstable).
- Gives gradient clipping as the remedy for one and explains why it cannot fix the other.
- Shows the additive cell-state update and its diagonal Jacobian.
- Mentions the forget-gate bias initialisation and that clipping is still used.

KEYWORDS TO WRITE IN THE EXAM

backpropagation through time · Jacobian product · spectral norm · vanishing gradient · exploding gradient · gradient clipping · cell state · forget gate · diagonal Jacobian · time constant

Q3 Compare standard, bidirectional, deep and LSTM recurrent networks

CO5 · Understand–Analyse · 10 marks · ~2.5 pages

Standard recurrent network

$$h_t = \tanh\left(W_{xh}x_t + W_{hh}h_{t-1} + b\right) \tag{5.12}$$

One direction, one layer, one state. The parameter count is quadratic in the hidden width, the memory is constant per step, and the geometric gradient decay of the previous answer applies. It is the reference point against which the other three are described.

Bidirectional recurrent network

Run two independent recurrences, one left-to-right and one right-to-left, and concatenate their states at every step:

$$\overrightarrow{h}_t = f\left(x_t, \overrightarrow{h}_{t-1}\right), \qquad \overleftarrow{h}_t = f\left(x_t, \overleftarrow{h}_{t+1}\right) \tag{5.13}$$

$$h_t = \left[\overrightarrow{h}_t; \overleftarrow{h}_t\right] \tag{5.14}$$

The representation at each step now depends on the whole sequence, not only on the past, which is exactly right for tasks where the answer is available at once: part-of-speech tagging, named-entity recognition, speech recognition over a completed utterance. The costs are that parameters double, that the entire sequence must be available before the first output (so it cannot be used in streaming or causal settings), and that a bidirectional layer cannot sit under a causal decoder.

Deep (stacked) recurrent network

Stack recurrent layers, the output sequence of one becoming the input sequence of the next:

$$h_t^{(l)} = f\left(W^{(l)} h_{t-1}^{(l)} + U^{(l)} h_t^{(l-1)} + b^{(l)}\right) \tag{5.15}$$

Depth buys a hierarchy over time: lower layers capture local patterns, higher layers capture longer structure, exactly as in a convolutional network but along the time axis. The costs are memory (all states at every layer must be kept for the backward pass), longer training, and a greater tendency to overfit on small data. Residual connections between layers are the standard stabiliser.

Long short-term memory

Keep the topology of a standard recurrence but change the cell to a gated one with a separate cell state:

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f), \qquad i_t = \sigma(W_i[h_{t-1}, x_t] + b_i) \tag{5.16}$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tanh\left(W_c[h_{t-1}, x_t] + b_c\right) \tag{5.17}$$

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o), \qquad h_t = o_t \odot \tanh\left(c_t\right) \tag{5.18}$$

The parameter count per layer rises to four times the product of the hidden width and the sum of hidden and input widths — four gates instead of one transition — and the memory roughly doubles. In exchange, the gradient path through the cell state is diagonal, and long-range dependencies become learnable. The gated recurrent unit is the lighter variant with two gates and no separate cell state, usually within a point of the LSTM's accuracy.

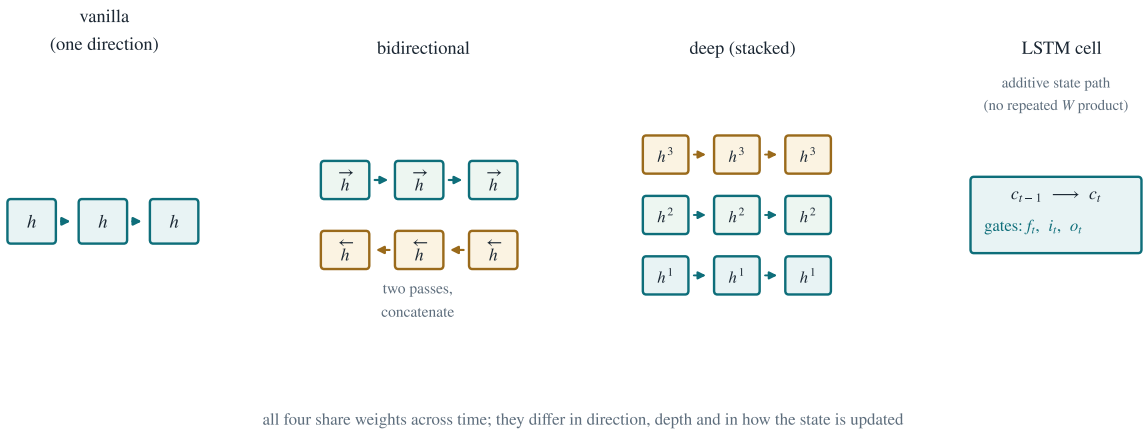


Fig. 5.3 The four topologies side by side: one direction, two directions, stacked layers, and the gated cell.

Comparison

Criterion	Standard	Bidirectional	Deep	LSTM
direction	forward only	both	forward (or both)	forward (or both)
parameters per layer	base	doubled	multiplied by depth	four times base
long dependencies	poor	poor	poor	good
needs the full sequence	no	yes	no	no
streaming or online use	yes	no	yes	yes
memory per step	low	doubled	times the depth	doubled and times the depth
typical use	toy models, short series	tagging, speech encoding	large corpora	general sequence modelling

Choosing

1. Short sequences, tight compute, or a baseline: a standard recurrence.
2. A completed sequence and a per-token label: bidirectional, and bidirectional *plus* gating if the dependencies are long.
3. A large corpus and a need for hierarchy: deep, usually with residual connections.

4. Long dependencies in a streaming setting: LSTM or gated recurrent unit.
5. Modern default: a gated stack, with attention if the sequence is long; a transformer if the dataset is large.

ANSWER STRUCTURE

Describe each variant by the one axis it changes — direction, depth, or the state update — then compare them in a table on parameters, memory and dependency length. That organisation answers the question directly and avoids a list of definitions.

GOLDEN LINE

The four models differ along three axes: direction (bidirectional), depth (stacked) and the state update (gated LSTM); only the last changes how far a gradient can travel, and only the bidirectional one requires the whole sequence in advance.

EXAMINER'S CHECKLIST

- Writes the recurrence for each variant.
- Assigns parameter counts and memory per variant.
- Explains what each axis buys and what it costs.
- Notes that the bidirectional model needs the entire sequence, so it is not causal.
- Ends with a selection rule.

KEYWORDS TO WRITE IN THE EXAM

recurrence · bidirectional · stacked RNN · LSTM · gated recurrent unit · parameter count · causal constraint · long-range dependency · residual connection

Q4 Explain the gates of an LSTM and the role of the internal cell state

CO5 · Understand–Analyse · 10 marks · ~3 pages

Why a gate at all

The simple recurrence overwrites its state at every step with a full matrix multiplication, so there is no way for the network to *decide* to keep something. A gate is a multiplicative, learned valve: a sigmoid produces a vector with entries between zero and one that scales another vector elementwise, and the network learns how far to open it. Three gates and one candidate constitute the LSTM.

The equations

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f) \tag{5.19}$$

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i) \tag{5.20}$$

$$\tilde{c}_t = \tanh(W_c[h_{t-1}, x_t] + b_c) \tag{5.21}$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \tag{5.22}$$

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o) \tag{5.23}$$

$$h_t = o_t \odot \tanh(c_t) \tag{5.24}$$

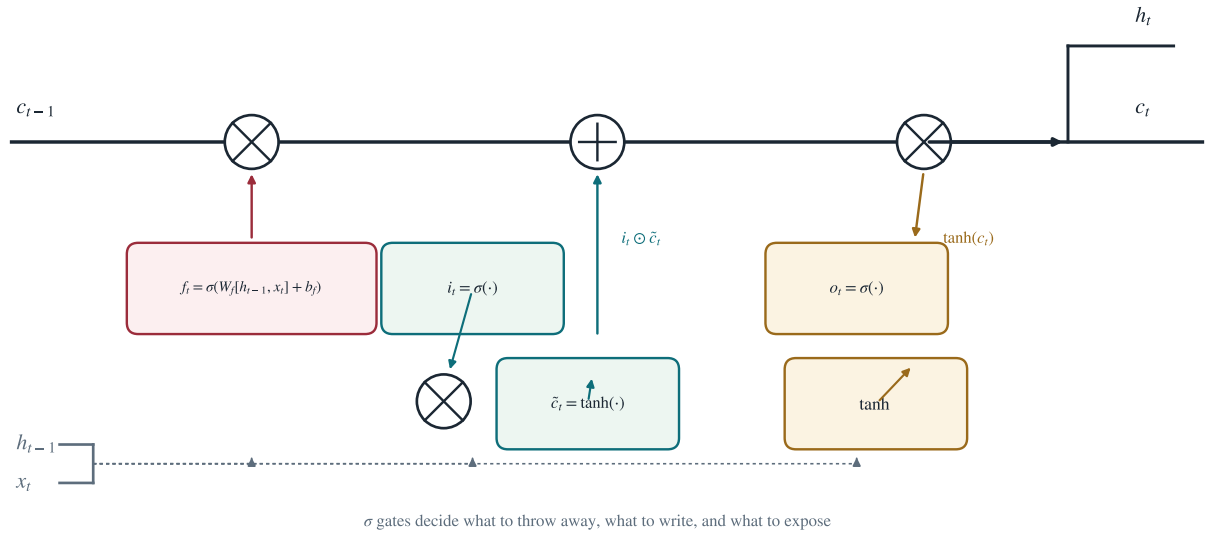


Fig. 5.4 The cell: the horizontal line at the top is the cell state, written by the input gate and read through the output gate, with the forget gate deciding what survives.

What each gate does

- **Forget gate.** Scales the previous cell state. A value near one keeps the memory, a value near zero erases it. It is the gate that gives the LSTM its long memory, and its bias is initialised positive so that memory is preserved at the start of training.
- **Input gate.** Scales the new candidate, deciding how much of the current input to write into the state. Together with the candidate it forms the "write" operation.
- **Candidate.** The actual content proposed by the current input and previous hidden state, squashed into the interval from minus one to one so that it can be added or subtracted.
- **Output gate.** Decides how much of the cell state to expose as the hidden output. This is the separation that a simple recurrence lacks: the internal memory and the external representation are different objects.

The internal cell state

The cell state runs along the top of the diagram, modified only by elementwise multiplication and addition:

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \tag{5.25}$$

Three properties follow directly from the plus sign:

1. **Additive, not multiplicative, update.** Adding is what makes the gradient path linear. The Jacobian is diagonal, so no direction is mixed and no matrix power appears.
2. **Per-channel memory.** Each coordinate of the state has its own forget gate, hence its own decay constant. The network can keep one feature for hundreds of steps and another for two.
3. **Constant error carousel.** With the forget gate at one and the input gate at zero the state is copied unchanged and the gradient is exactly one — the property that made LSTMs able to learn dependencies of hundreds of steps at all.

The hidden state is the *interface* to the rest of the network: it is bounded by the tanh, so it can be fed forward or into a classifier without scaling problems, and it can choose to reveal only part of the memory.

Gates as learned control

It is worth stating what the gates are *not*: they are not attention weights over positions, and they do not see the future. They are functions of the current input and the previous hidden state, so all their decisions are causal. Reading a gated cell as a small differentiable program — keep, write, read — is the most compact correct mental model.

COMMON EXAM SLIP

Writing the output as the output gate times the cell state, without the tanh. The tanh is not cosmetic: it bounds the exposed state, and it is part of why the output layer stays well conditioned.

GOLDEN LINE

The forget gate decides what survives, the input gate what is written, the output gate what is exposed, and the additive cell-state path is what turns a product of dense Jacobians into a product of diagonal gates.

EXAMINER'S CHECKLIST

- Writes all six equations with the gate ranges.
- Explains each gate as a multiplicative valve with a role.
- Explains the diagonal Jacobian and the constant error carousel.
- Distinguishes the cell state from the hidden state and says why the tanh matters.
- Notes causality: the gates see only past and present.

KEYWORDS TO WRITE IN THE EXAM

forget gate · input gate · output gate · candidate state · cell state · hidden state · sigmoid · constant error carousel · diagonal Jacobian · gating

Q5 A simple RNN fails on very long sequences — give a complete strategy

CO5 · Apply–Create · 10 marks · ~3 pages

Diagnose the failure first

"Fails on long sequences" has four distinct causes, and the remedy differs for each. Say which one you are fixing:

Symptom	Cause	Remedy family
cannot learn a dependency far back	vanishing gradient	gating, attention
loss spikes then becomes not-a-number	exploding gradient	clipping, smaller step
trains but memory overflows	the backward pass stores every state	truncated backpropagation
good on short, bad on long test sequences	distribution shift, no length generalisation	curriculum, chunking

The strategy, in order of cost

- ***Step 1 · Clip and truncate***

Backpropagation through time needs every state of the sequence in memory, so cost grows linearly in the length and quadratically in the backward pass through time. **Truncated backpropagation** backpropagates only a fixed number of steps (typically 32–128), carrying the state forward without gradient across chunk boundaries. This bounds memory and is the standard engineering answer for long audio or sensor streams. Combine it with gradient-norm clipping, which is not optional for recurrent nets.

- ***Step 2 · Change the cell***

Replace the simple recurrence by a gated one — LSTM or gated recurrent unit — so that the gradient path is a product of diagonal gates rather than of the same dense matrix. Initialise the forget-gate bias to one and use orthogonal initialisation for the recurrent weights, which keeps the initial Jacobian's singular values near one. This is the single highest-value change when the failure is a long-range dependency.

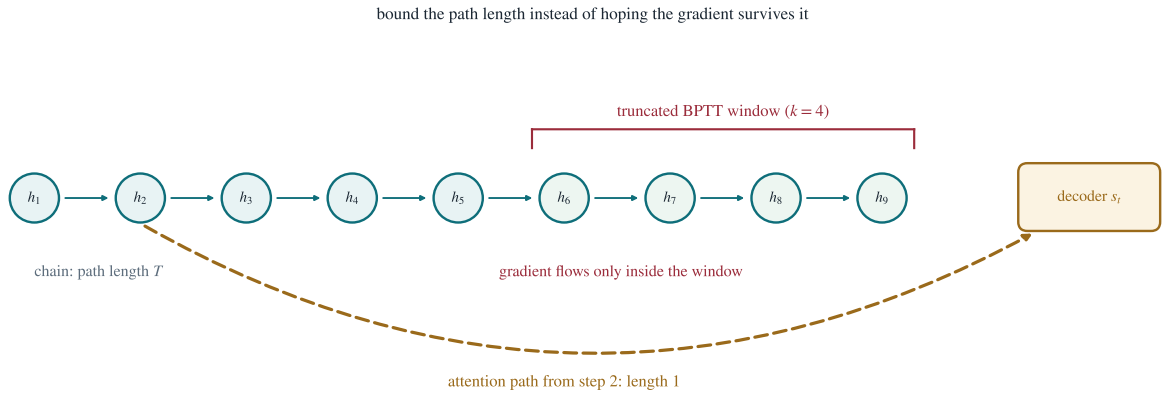


Fig. 5.5 Two ways to shorten the path: bound the window over which gradients flow, or let the decoder reach any step directly.

Step 3 · Normalise and schedule

Layer normalisation inside the recurrence stabilises the state statistics across time steps; a learning-rate warm-up followed by a decay schedule prevents early divergence; noise or dropout applied to the *inputs and outputs* of the recurrent layer, not to the recurrent connections, regularises without breaking the memory.

Step 4 · Restructure the problem

- **Downsample or chunk the input.** Ten minutes of audio at 16 kHz is almost ten million steps; frame-level features at 100 Hz reduce it by two orders of magnitude, and a hierarchical model (frames, then phones, then words) reduces the dependency length the recurrence must span.
- **Stateful batching.** Keep the hidden state across consecutive batches so that the model effectively sees the whole stream, while each batch is trained independently.
- **Sliding windows with overlap** for inference, so that no prediction depends on a context the model never saw.

Step 5 · Add attention

When the dependency really is long, no recurrence keeps it reliably. Attention computes each output directly from a weighted sum of *all* encoder states,

$$\alpha_{t,s} = \frac{\exp(e_{t,s})}{\sum_{s'} \exp(e_{t,s'})}, \qquad e_{t,s} = v^{\top} \tanh(W_h h_s + W_s s_{t-1}) \tag{5.26}$$

so the path length between any two positions is one, not the length of the sequence. This removes the gradient decay entirely at the cost of compute quadratic in the length, and it is the reason transformers replaced recurrent models for long sequences.

Step 6 · Replace the recurrence

If the sequence is long, the data are plentiful and latency is not critical, a transformer or a state-space model is the modern default. Say this as a recommendation, not as a dismissal: recurrent models remain the right choice for streaming inference with constant memory.

> **THE SHAPE OF A GOOD ANSWER**
>
> The examiner wants an ordered plan, not a list of tricks. Write: *clip, gate, normalise, restructure, attend*, and against each step the symptom it fixes and the cost it adds. Then state the decision rule: if the sequence fits in memory and the dependency is under a few hundred steps, a gated model with truncated backpropagation suffices; beyond that, attention is the answer.

GOLDEN LINE

For long sequences: bound the memory with truncated backpropagation, bound the step with clipping, replace the cell by a gated one, restructure with hierarchical or stateful processing, and if the dependency is still too long, use attention so that the path length stops depending on the sequence.

EXAMINER'S CHECKLIST

- Separates the four failure causes before prescribing.
- Explains truncated backpropagation and what it gives up.
- Justifies the gated cell by the diagonal Jacobian.
- Gives at least two restructurings: hierarchy, stateful batching.
- Ends with attention and states its cost.

KEYWORDS TO WRITE IN THE EXAM

truncated backpropagation · gradient clipping · orthogonal initialisation · layer normalisation · hierarchical model · stateful batching · attention · path length · quadratic cost

Q6 Design an encoder–decoder sequence-to-sequence model and derive its training objective

CO5 · Apply–Analyse · 10 marks · ~3 pages

The problem it solves

A single recurrent network maps a sequence to a label, or to a sequence of the same length. Translation, summarisation and speech transcription map a sequence to a sequence of a **different** length and a different vocabulary. The encoder–decoder architecture handles this by separating "read" from "write".

The encoder

A recurrent network consumes the source sequence and produces a hidden state per step; the final state is the context vector:

$$h_t = f_{enc}(h_{t-1}, x_t), \qquad c = h_T \tag{5.27}$$

With a bidirectional encoder the context is the concatenation of the two final states, so the summary sees the sentence from both ends.

The decoder

A second recurrent network is initialised from the context and generates the target one token at a time, conditioned on the context and on what it has already produced:

$$s_{t'} = f_{dec}(s_{t'-1}, y_{t'-1}, c) \tag{5.28}$$

$$P(y_{t'} \mid y_{<t'}, x) = \mathrm{softmax}(W_o s_{t'} + b_o) \tag{5.29}$$

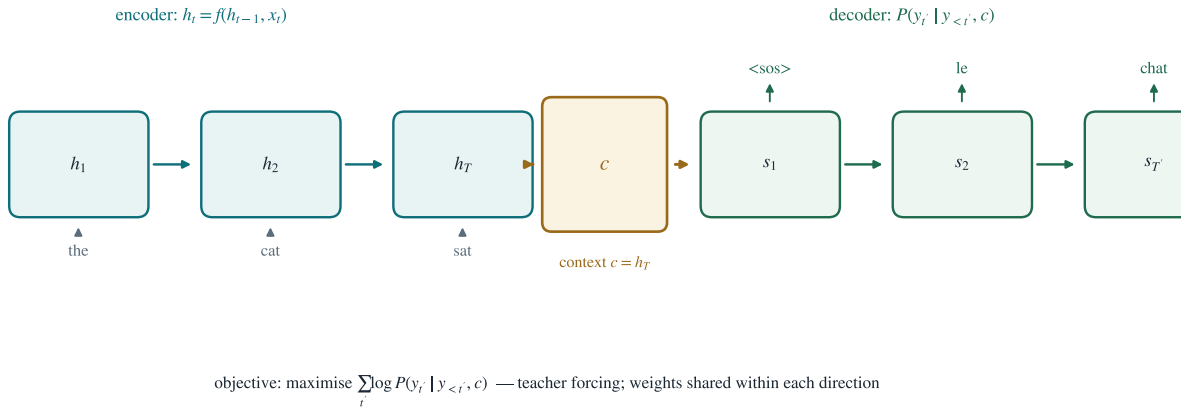


Fig. 5.6 Encoder compresses the source into the context vector; the decoder unrolls the target conditioned on it and on its own previous output.

The training objective

The model defines a conditional distribution over whole target sequences by the chain rule:

$$P(y \mid x) = \prod_{t'=1}^{T'} P(y_{t'} \mid y_{<t'}, x) \tag{5.30}$$

Training maximises the log-likelihood of the reference translation — equivalently minimises the negative log-likelihood, summed over the corpus:

$$\mathcal{L}(\theta) = -\sum_{n=1}^{N} \sum_{t'=1}^{T'_n} \log P_\theta\left(y_{t'}^{(n)} \mid y_{<t'}^{(n)}, x^{(n)}\right) \tag{5.31}$$

This is **cross-entropy with the previous target tokens as input**, and it factorises exactly like a language model; the only difference is that every conditional is also conditioned on the context. Because each factor is a softmax, the gradient with respect to the decoder logits is again predicted minus target probability, and the whole model is trained by backpropagation through both networks.

Teacher forcing and its consequence

During training the previous target token is the *true* one, not the model's own prediction — this is **teacher forcing**. It makes every step's gradient well defined and allows the computation to be parallel over the target positions within a sequence. The mismatch is that at inference the model must consume its own outputs, which it has never seen during training; the resulting error accumulation is called **exposure bias**. Remedies: scheduled sampling (feed the model's own output with a probability that increases over training), beam search at inference, and sequence-level objectives such as minimum risk training.

Inference

Greedy decoding takes the most probable token at each step; beam search keeps the best few partial sequences and maximises

$$\hat{y} = \arg\max_{y} \sum_{t'} \log P(y_{t'} \mid y_{<t'}, x) \tag{5.32}$$

which is a better approximation of the most probable sequence but does not guarantee it, since the most probable sequence is not the sequence of most probable tokens.

The bottleneck and the fix

A single fixed-length context vector must carry the entire source, so performance degrades with length. **Attention** lets the decoder look at all encoder states with weights recomputed at every step:

$$c_{t'} = \sum_s \alpha_{t',s} h_s, \qquad \alpha_{t',s} = \mathrm{softmax}_s(e_{t',s}) \tag{5.33}$$

so the decoder can align to the relevant part of the input. This is the same mechanism that, without the recurrence, becomes the transformer.

TWO DERIVATIONS TO KEEP STRAIGHT

The chain rule gives a product over *output* positions, each conditioned on the prefix. The negative log turns the product into a sum over positions, which is why the loss adds over time rather than multiplying.

GOLDEN LINE

An encoder compresses the source into a context vector and a decoder factorises the target distribution autoregressively, conditioned on that vector, trained by maximising log-likelihood under teacher forcing.

EXAMINER'S CHECKLIST

- Describes the encoder, the context vector and the decoder separately.
- Writes the factorisation and the negative log-likelihood loss over a corpus.
- Explains teacher forcing and exposure bias.
- Distinguishes greedy from beam search.
- Identifies the fixed-context bottleneck and gives attention as the fix.

KEYWORDS TO WRITE IN THE EXAM

encoder–decoder · context vector · autoregressive factorisation · teacher forcing · exposure bias · cross-entropy over time · beam search · attention alignment · sequence transduction

Q7 Design an autoencoder for dimensionality reduction — bottleneck, objective, and how to avoid the identity mapping

CO5 · Apply–Create · 10 marks · ~3 pages

The architecture

An autoencoder is two networks trained together: an encoder that maps the input to a low-dimensional code, and a decoder that reconstructs the input from the code.

$$z = E_\phi(x) \in \mathbb{R}^d, \qquad \hat{x} = D_\theta(z), \qquad d \ll n \tag{5.34}$$

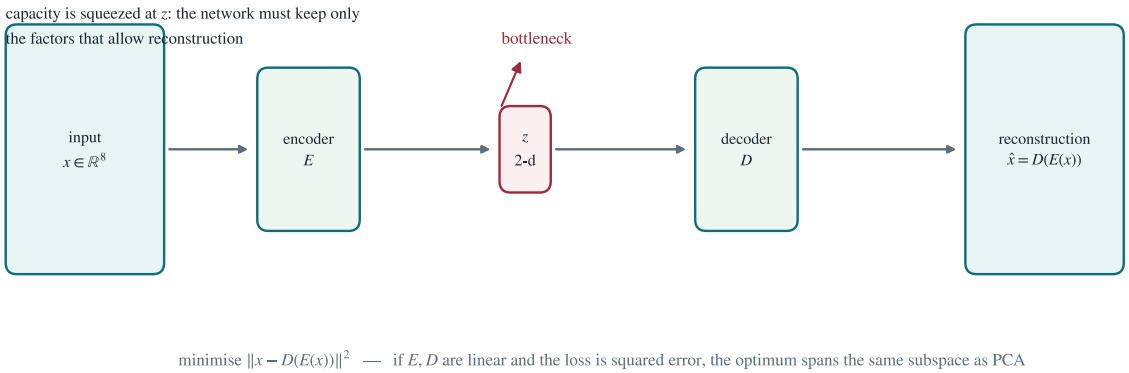


Fig. 5.7 The bottleneck is the only constraint that forces compression; without it the network can learn the identity.

The objective is the reconstruction error, averaged over the dataset:

$$\mathcal{L}(\phi, \theta) = \frac{1}{m} \sum_{i=1}^{m} \left\| x^{(i)} - D_\theta\left(E_\phi(x^{(i)})\right) \right\|^2 \tag{5.35}$$

For binary or normalised inputs the cross-entropy form is used instead. Training is ordinary backpropagation through both halves; no labels are needed, which is why the method is unsupervised.

Why the bottleneck matters

If the code is at least as wide as the input and both halves are expressive, the trivial solution — the composition being the identity — attains zero loss and the code is useless. The bottleneck is therefore not an implementation detail: it is the constraint that creates the representation. Three ways to enforce it:

1. **Undercomplete** (a code narrower than the input): the code cannot hold the input, so the network must keep the directions of greatest variation.
2. **Sparse** (a wide code with a penalty on its activity): few units are active per input.
3. **Denoising**: corrupt the input and require the clean input back, which prevents copying even when the code is wide.

The linear case is exactly PCA

Consider a linear encoder and a linear decoder with squared error. The optimum of the reconstruction objective is attained when the composition of the two matrices is the orthogonal projector onto the top-d principal subspace of the data covariance. The argument: the composition is a rank-d matrix, and the squared reconstruction error of a rank-d projector is the sum of the discarded eigenvalues, minimised by keeping the d largest. The decoder's columns then span the same subspace as the first d principal components, and if they are constrained to be orthonormal the autoencoder recovers PCA exactly.

The corollary is the examinable statement: *a linear autoencoder with a linear decoder and squared error is PCA with extra parameters; the value of the autoencoder is that it generalises PCA to nonlinear manifolds.*

Design choices that matter

Choice	Recommendation	Reason
code size	sweep; pick the elbow of reconstruction error	same role as the number of components in PCA
depth	two to four layers per half	enough nonlinearity, still trainable
activation	ReLU in the encoder, linear or sigmoid at the output	match the input range
loss	squared error for continuous, cross-entropy for probabilities	matches the noise model
bottleneck unit	linear for a "PCA-like" code	keeps the geometry interpretable
regularisation	denoising or sparsity penalty	prevents identity, improves features
validation	held-out reconstruction error	the only honest signal without labels

Using it for dimensionality reduction

Train on the full dataset (or on a training split and apply to the rest), then discard the decoder and keep the code as the reduced representation. Two practical points:

1. **Standardise first** — otherwise the first principal direction is dominated by the variable with the largest units.
2. **Compare with PCA at the same code size**: if the data are close to linear, PCA wins on speed and interpretability; autoencoders win when the data lie on a curved manifold, which is typical for images and spectra.

THE SINGLE SENTENCE THAT EARNS THE MARK

"A linear autoencoder with squared error spans the PCA subspace; the nonlinear version is a generalisation of PCA to curved manifolds, and the bottleneck is what forces it to learn the manifold rather than the identity."

GOLDEN LINE

An autoencoder minimises the squared reconstruction error under a bottleneck; the bottleneck, or a denoising or sparsity constraint, is what prevents the identity map, and in the linear squared-error case the learned subspace is exactly the PCA subspace.

EXAMINER'S CHECKLIST

- Draws the encoder, bottleneck and decoder, and writes the loss.
- Explains why the code must be narrower and what the identity solution is.
- Names two other ways to prevent triviality: sparsity, denoising.
- Derives or states the linear-autoencoder / PCA equivalence.
- Gives practical choices: standardisation, code-size sweep, comparison with PCA.

KEYWORDS TO WRITE IN THE EXAM

autoencoder · bottleneck · undercomplete · reconstruction loss · identity mapping · denoising autoencoder · sparse coding · PCA equivalence · manifold learning

Q8 How do autoencoders learn representations, and when are those representations useful?

CO5 · Understand–Evaluate · 10 marks · ~2.5 pages

What is learned, and why it is not just compression

The encoder must map every input to a code from which the decoder can rebuild it. Any information the encoder throws away is lost for every input, so it is forced to keep the *factors that vary systematically* and discard the noise that is specific to each example. That selection is what makes the code a representation rather than a compression: the network is learning the generative factors of the data, because those are what reconstruction requires.

Formally, the autoencoder learns a manifold: if the data concentrate near a low-dimensional surface, then a narrow code can reconstruct them only if the encoder charts that surface and the decoder inverts the chart. The code is then a coordinate system on the data manifold.

Three ways to shape the representation

1. **Denoising autoencoder.** Corrupt the input and minimise the error against the clean original. The network cannot copy, so it must learn the structure that allows recovery. This is the most reliable way to get useful features from a wide code.
2. **Sparse autoencoder.** Penalise the activity of the code so that each input activates few units. The units become more selective and closer to the "one concept per unit" ideal; interpretability improves.
3. **Contractive or variational.** A contractive penalty on the encoder Jacobian makes the code insensitive to small input perturbations; a variational autoencoder makes the encoder output a distribution and trains with the evidence lower bound, which additionally yields a usable generative model.

When the representation is useful

Use	Why the code helps	Caveat
downstream classifier with few labels	features trained on all data, labelled or not	the code may discard label-relevant detail
visualisation and clustering	low-dimensional coordinates	clusters are of the data, not of the classes
denoising and inpainting	projection onto the manifold	blurry for large corruptions
anomaly detection	unusual inputs reconstruct badly	the threshold needs calibration
initialisation for a deep net	features start in a good region	usually beaten by supervised pretraining
generative modelling	sampling in code space	posterior collapse in variational models

The key caveat is stated in the table's first row: an autoencoder is trained to preserve *everything* that is reconstructible, whereas a classifier needs only what separates the classes. A representation that is excellent for reconstruction can be poor for classification — the classic counter-example is a code that keeps the background texture and discards the object. This is why representation learning moved toward objectives that are *discriminative by construction*: contrastive learning, masked-token prediction, and their descendants.

Evaluating a learned representation

Three honest tests, in increasing order of cost:

1. **Linear probe.** Freeze the encoder and train a linear classifier on the code; accuracy measures how linearly separable the concepts have become.
2. **Transfer.** Fine-tune on a small labelled set and compare with training from scratch on the same budget.
3. **Task-aligned reconstruction.** Check that improved reconstruction actually improves the task; often it does not, and the gap is the evidence that the objective is misaligned.

THE OVER-COMPLETE TRAP

If the code is at least as wide as the input and nothing else constrains it, the autoencoder learns the identity and the "representation" is the input. Any claim about learned features must therefore come with the mechanism that prevented copying: a bottleneck, a sparsity penalty, or noise.

GOLDEN LINE

An autoencoder learns the generative factors of the data because reconstruction forces it to keep what varies systematically and drop what is idiosyncratic; the representation is useful when it is constrained (bottleneck, sparsity, noise) and when the downstream task needs what reconstruction preserves.

EXAMINER'S CHECKLIST

- Explains representation learning as manifold charting, not compression.
- Describes denoising, sparse and variational variants and what each changes.
- Lists downstream uses with a caveat for each.
- States the reconstruction-versus-discrimination mismatch.
- Gives concrete evaluation: linear probe, transfer, task-aligned reconstruction.

KEYWORDS TO WRITE IN THE EXAM

representation learning · manifold · generative factors · denoising autoencoder · sparsity · contractive penalty · variational autoencoder · evidence lower bound · linear probe · transfer learning

Appendix A · Formula sheet

Everything you are expected to write down, on three pages. Derivations are in the answers; this is the skeleton.

UNIT III — regularisation and optimisation

Quantity	Expression
Regularised objective	$\tilde{J}(\theta) = J(\theta) + \lambda \Omega(\theta)$
Weight decay update	$\theta \leftarrow (1 - \eta\lambda)\theta - \eta \nabla J(\theta)$
Penalty gradients	$\nabla J + \lambda\theta \quad (L_2); \qquad \nabla J + \lambda\, \mathrm{sgn}(\theta) \quad (L_1)$
Soft threshold	$w^\star = \mathrm{sign}(y)\max(\lvert y\rvert - \lambda, 0)$
Ridge closed form	$\hat{\theta} = (X^\top X + \lambda I)^{-1} X^\top y$
Elastic net	$\Omega = \lambda_1 \lVert\theta\rVert_1 + \lambda_2 \lVert\theta\rVert_2^2$
Momentum	$v_t = \mu v_{t-1} - \eta g_t; \quad \theta_{t+1} = \theta_t + v_t$
Adam step	$\theta \leftarrow \theta - \eta\, \hat{m}_t / (\sqrt{\hat{s}_t} + \varepsilon)$
Bias correction	$\hat{m}_t = m_t / (1 - \beta_1^t)$
He initialisation	$W \sim \mathcal{N}(0,\ 2/n_{in})$
Early stopping as L2	$\lambda \approx 1/(\tau\eta)$
Dropout at test time	$W_{test} = q\, W; \qquad \tilde{h} = m \odot h / q$

UNIT IV — convolutional networks

Quantity	Expression
Output size	$O = \lfloor (W - K + 2P)/S \rfloor + 1$
Parameters	$K^2 C_{in} C_{out} + C_{out}$
Cost	$2\, HWK^2 C_{in} C_{out}$ FLOPs
Receptive field	$r_l = r_{l-1} + (K_l - 1)\Pi_{i<l} S_i$
Dilation	$K_{eff} = K + (K-1)(d-1)$
Same padding	$P = \lfloor K/2 \rfloor, \ \ S = 1$
Global average pool	$g_c = 1/HW \Sigma_{i,j} A_{i,j,c}$
Softmax + cross-entropy	$\partial L / \partial z_c = \hat{y}_c - y_c$
Class activation map	$M_c(i,j) = \Sigma_k w_{k,c} A_{i,j,k}$

UNIT V — recurrent networks and autoencoders

Quantity	Expression
Simple RNN	$h_t = \tanh(W_{xh} x_t + W_{hh} h_{t-1} + b)$
BPTT Jacobian	$\partial h_i / \partial h_{i-1} = \mathrm{diag}(1 - h_i^2) W_{hh}$
Gradient bound	$\lVert \partial L_t / \partial h_k \rVert \le C \gamma^{t-k}$
Clipping	$g \leftarrow g \cdot \min(1,\ c/\lVert g\rVert)$
LSTM state	$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$
LSTM parameters	$4\, d_h (d_h + d_x)$
Seq2seq likelihood	$P(y \mid x) = \Pi_t P(y_t \mid y_{<t}, x)$

Quantity	Expression
Attention	$\alpha_{t,s} = \mathrm{softmax}_s(e_{t,s}); \quad c_t = \Sigma_s \alpha_{t,s} h_s$
Autoencoder loss	$\|x - D(E(x))\|^2$

Appendix B · Writing a ten-mark answer

Ten marks are not awarded for volume. They are awarded for the sequence *definition mechanism derivation comparison consequence*, with the notation fixed in the first two lines. The template below is the one used throughout this book; it fits any of the twenty-four questions.

Mark	What the examiner looks for	Where it appears in this book
2	A precise definition with the symbols declared	the opening paragraph
3	The mechanism or derivation, one numbered equation per step	the second section
2	A worked number, a figure or a table	the middle of the answer
2	A comparison or an evaluation with trade-offs	the closing table
1	A conclusion in one sentence	the golden line

Four habits that cost marks when missing.

- Declare your symbols. W, θ, λ and t must mean one thing each, from the first use.
- Answer the question that was asked. 'Underfitting caused by pooling' is not 'overfitting, so add dropout'.
- Put numbers in. A parameter count, a FLOP count or an output size is worth more than an adjective.
- Finish with the conclusion, not with a table. The last sentence is what the examiner remembers.

The order in which to spend twenty minutes

Minutes	Do this
0–2	Write the definition and the symbols. Do not start with prose.
2–10	Write the derivation, one equation per line, numbered.
10–15	Draw the figure or write the numeric example.
15–18	Write the comparison table — three rows is enough.
18–20	Write the golden line and the keywords.

Notation used throughout

Symbol	Meaning
m, n	number of examples; dimension of the input
θ, W, b	the parameter vector; the weight matrix and bias of a layer
η, λ	learning rate; regularisation strength (the KKT multiplier)
K, S, P	kernel size, stride and padding of a convolution
H, W, C	height, width and channel count of a feature map
T	sequence length, in time steps
h, c	hidden state and cell state of a recurrent layer
L, J	the loss or objective being minimised

Rapid revision grid

If you have ten minutes before the paper, read only these pages.

UNIT III

Q	The one line you must write	Trap to avoid
1	Diagnose before you regularise: plot the training and validation curves; a wide gap is variance, both high is bias.	Do not answer with 'add dropout' alone — the question asks for a plan, so order the interventions and say how you will measure success.
2	L_1 gives sparse weights because the derivative of $\lvert w \rvert$ is a constant sign, so a fixed threshold can reach zero; L_2 shrinks proportionally and never reaches it.	Sparsity is a property of the geometry (corner of the diamond), not of the formula 'absolute value'.
3	Dropout = 2^n weight-sharing sub-networks; at test time multiply weights by the keep probability q.	Forgetting inverted dropout: with it you divide by q during training and do nothing at test time.
4	Constrained problem, Lagrangian, stationarity, complementary slackness: if the constraint is active then the multiplier is the penalty coefficient.	Claiming the two problems are identical for the same λ — they are equivalent only through the map between λ and the radius t.
5	Initialisation → momentum → RMSProp → Adam, each rung fixing a specific failure: scale, ill-conditioning, per-parameter scaling.	Presenting Adam as a cure-all; it still needs a sane learning rate, warm-up and clipping.
6	Deep learning optimises a surrogate, with sampled gradients, on a non-convex surface dominated by saddle points.	Saying 'local minima are the problem' — in high dimensions local minima are rare and usually good; plateaus and saddle points hurt more.
7	AdaGrad accumulates from the start and stalls; RMSProp forgets; Adam forgets and adds momentum with bias correction.	Forgetting Adam's bias correction of both moving averages.
8	Early stopping is free and needs a validation set; dropout costs epochs; augmentation costs compute but also enlarges the effective data.	Using the test set to decide when to stop.

UNIT IV

Q	The one line you must write	Trap to avoid
1	A CNN is a stack that shrinks spatial size and grows channel count, ending in global average pooling and a small softmax.	Forgetting to say how the kernel size and the receptive field relate to the object size in the data.
2	Output size by the floor formula; FLOPs are twice the product of output size, kernel area and both channel counts.	Mixing FLOPs with multiply–accumulate operations, or forgetting that padding must preserve the size when the stride is one.
3	Local connectivity plus weight sharing: the same filter slides over the image, so parameters are independent of the image size.	Saying a CNN 'has fewer layers' — it does not; it has fewer parameters.
4	Convolution encodes translation equivariance and locality; pooling encodes small-shift tolerance.	Claiming a CNN is fully translation invariant — it is equivariant, and only approximately invariant after pooling.
5	Max-pooling is compatible with 'is there a cat' and incompatible with 'where is the cat'.	Answering 'use more pooling' — the question asks for the opposite.
6	Flatten gives height × width × channels inputs to the dense layer; GAP gives one average per channel.	Forgetting that GAP is what makes the network input-size agnostic.
7	Excess pooling removes the fine detail the label depends on; replace it with stride-1 or dilated convolutions.	Reducing capacity is the wrong reflex when the model is already underfitting.
8	Justify each block by a property of the data, then by the compute budget.	Quoting a famous architecture without connecting it to the problem's resolution, invariance or class count.

UNIT V

Q	The one line you must write	Trap to avoid
1	Unrolling replaces the loop by repeated copies of one cell with tied weights, so parameters are constant in the length but the graph has depth T.	Forgetting that the same weights are used at every step — that is the whole reason backpropagation through time accumulates into one matrix.
2	The gradient contains a product of T Jacobians; its norm shrinks or grows geometrically with the distance.	Saying the LSTM 'removes' the product — it replaces the repeated matrix by a diagonal gate path, which is not the same thing.
3	Bidirectional doubles the parameters and needs the whole sequence; depth stacks layers; the LSTM changes the cell, not the topology.	Confusing 'deep' with 'long': depth is over layers, length is over time.
4	The forget gate acts on the cell line, the input gate writes, the output gate exposes a squashed copy of the state.	Forgetting that the cell state is not the hidden output; the output gate is what separates them.
5	Truncated backpropagation, clipping, gating, restructuring, attention — in that order of cost.	Training on sequences longer than memory allows without truncation.
6	The decoder factorises the joint probability autoregressively, conditioned on the context vector.	Training with the model's own predictions instead of teacher forcing, then wondering why it diverges.
7	A bottleneck forces compression; without a bottleneck the network learns the identity.	Claiming an autoencoder is 'the same as' PCA in general — only the linear, squared-error case coincides.
8	Tie the encoder to a downstream task, or use denoising to force useful features.	Assuming any autoencoder produces transferable features; an over-complete one often memorises.

LAST THING BEFORE YOU ENTER THE HALL

Six sentences: regularise by diagnosis, not by reflex; a penalty is a constraint seen through a multiplier; a convolution is locality plus sharing; a pool is an invariance bought with position; a recurrence is a matrix applied T times, which is why it needs a gate; an autoencoder is a bottleneck that learns the manifold.